

Course: Cloud and Network Security CNS-C2-2026

Student Name: James Kirika

Student No: CS-CNS12-26028

September 28, 2026

Week 3 Assignment 2:

HTB Academy: Web Requests

Objectives.....	4
Introduction	4
Part 1: HTTP Fundamentals.....	4
Basic HTTP request to any URL by using it as an argument for cURL	4
cURL to download a page or a file and output the content into a file using the -O flag...5	
-h flag (curl help function)	6
Quiz: Get the flag	6
-k flag.....	7
HTTP Requests and Responses	7
Exercise: The -vvv flag shows	8
Browser DevTools.....	8
Quiz	8
HTTP Headers.....	9
cURL response headers.....	10
Exercise	12
Browser DevTools.....	12
Part 2: HTTP Methods	13
HTTP Methods and Codes.....	13
Status Codes.....	14
HTTP Basic Auth	14
Exercise: Try to view the response headers by adding -i	15
HTTP Authorization Header	16
Adding -v command.....	16
-H flag.....	17
GET Parameters.....	18
Quiz	18
POST parameter	18
Authenticated Cookies.....	19
JSON Data.....	21
Exercise:	22
Executing fetch on the console	24
Quiz	24

CRUD API	25
Read	25
Create	26
Update	27
Delete	28
Quiz	29
Conclusion	30
Links	30

Objectives

Part 1: HTTP Fundamentals

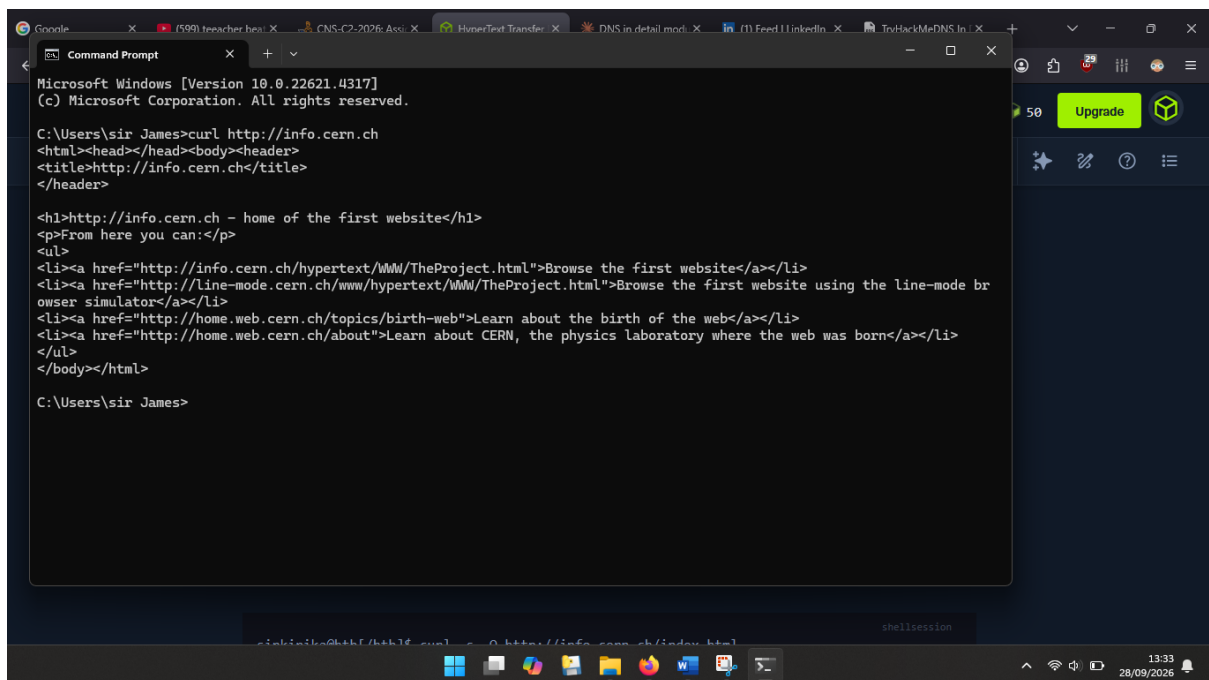
Part 2: HTTP Methods

Introduction

This report documents the HTB Academy "Web Requests" module, covering HTTP and HTTPS fundamentals, request/response headers, HTTP methods and status codes, Basic Authentication, and interacting with a REST API using cURL and browser DevTools.

Part 1: HTTP Fundamentals

Basic HTTP request to any URL by using it as an argument for cURL



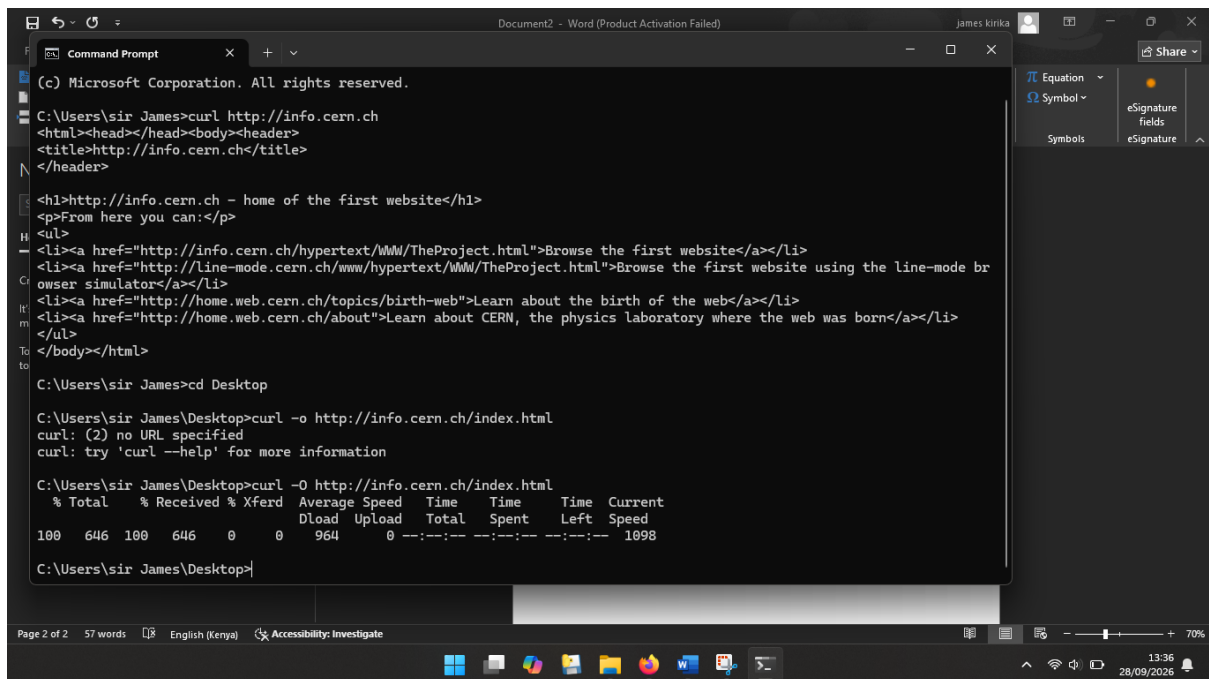
```
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sir James>curl http://info.cern.ch
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode browser simulator</a></li>
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>
</ul>
</body></html>

C:\Users\sir James>
```

cURL to download a page or a file and output the content into a file using the -O flag.



```
(c) Microsoft Corporation. All rights reserved.

C:\Users\sir James>curl http://info.cern.ch
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode browser simulator</a></li>
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>
</ul>
</body></html>

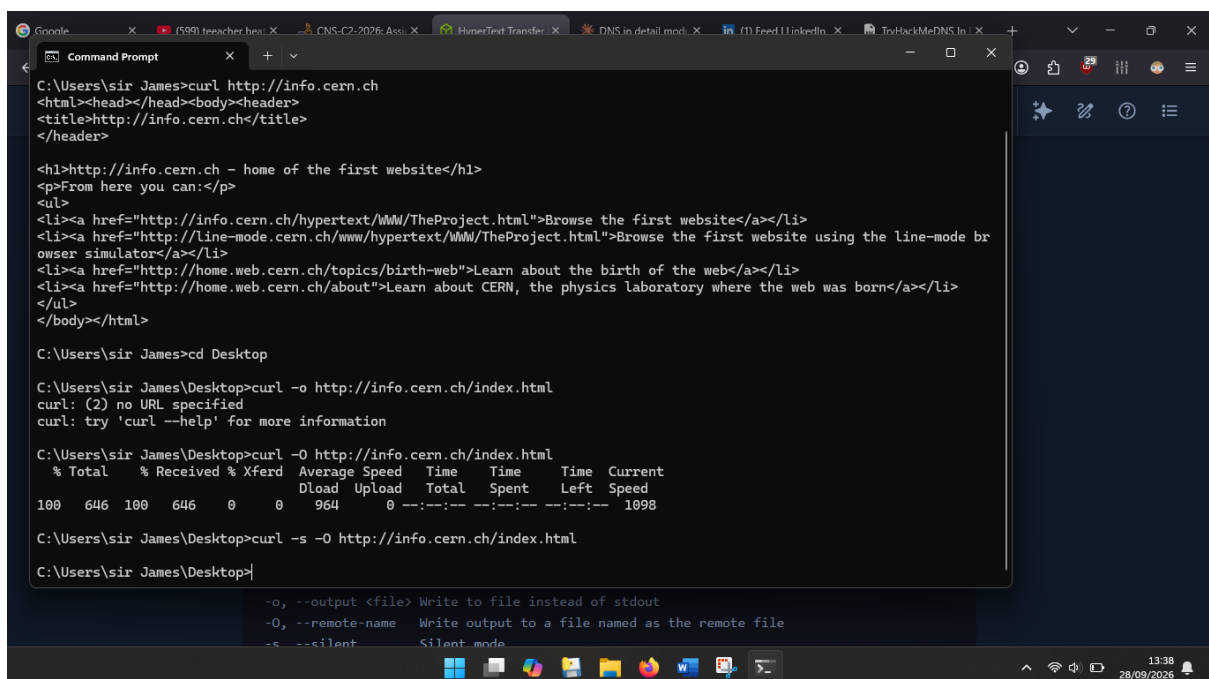
C:\Users\sir James>cd Desktop

C:\Users\sir James\Desktop>curl -o http://info.cern.ch/index.html
curl: (2) no URL specified
curl: try 'curl --help' for more information

C:\Users\sir James\Desktop>curl -O http://info.cern.ch/index.html
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 646    100 646    0    0   964    0 --:--:-- --:--:-- --:--:-- 1098

C:\Users\sir James\Desktop>
```

As we can see, the output was not printed this time but rather saved into index.html. We noticed that cURL still printed some status while processing the request. We can silent the status with the -s flag, as follows:



```
C:\Users\sir James>curl http://info.cern.ch
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode browser simulator</a></li>
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>
</ul>
</body></html>

C:\Users\sir James>cd Desktop

C:\Users\sir James\Desktop>curl -o http://info.cern.ch/index.html
curl: (2) no URL specified
curl: try 'curl --help' for more information

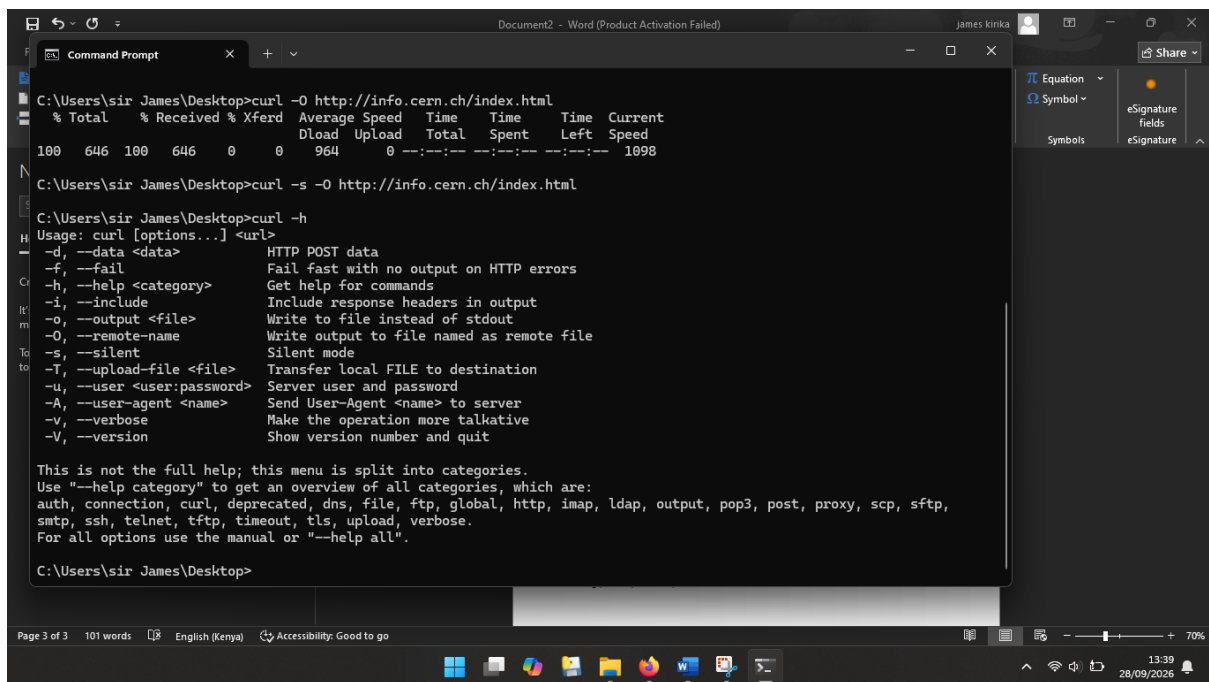
C:\Users\sir James\Desktop>curl -O http://info.cern.ch/index.html
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 646    100 646    0    0   964    0 --:--:-- --:--:-- --:--:-- 1098

C:\Users\sir James\Desktop>curl -s -O http://info.cern.ch/index.html
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode browser simulator</a></li>
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>
</ul>
</body></html>

C:\Users\sir James\Desktop>
```

-h flag (curl help function)



```
C:\Users\sir James\Desktop>curl -O http://info.cern.ch/index.html
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 646 100 646 0 0 964 0 --:--:-- --:--:-- --:--:-- 1098

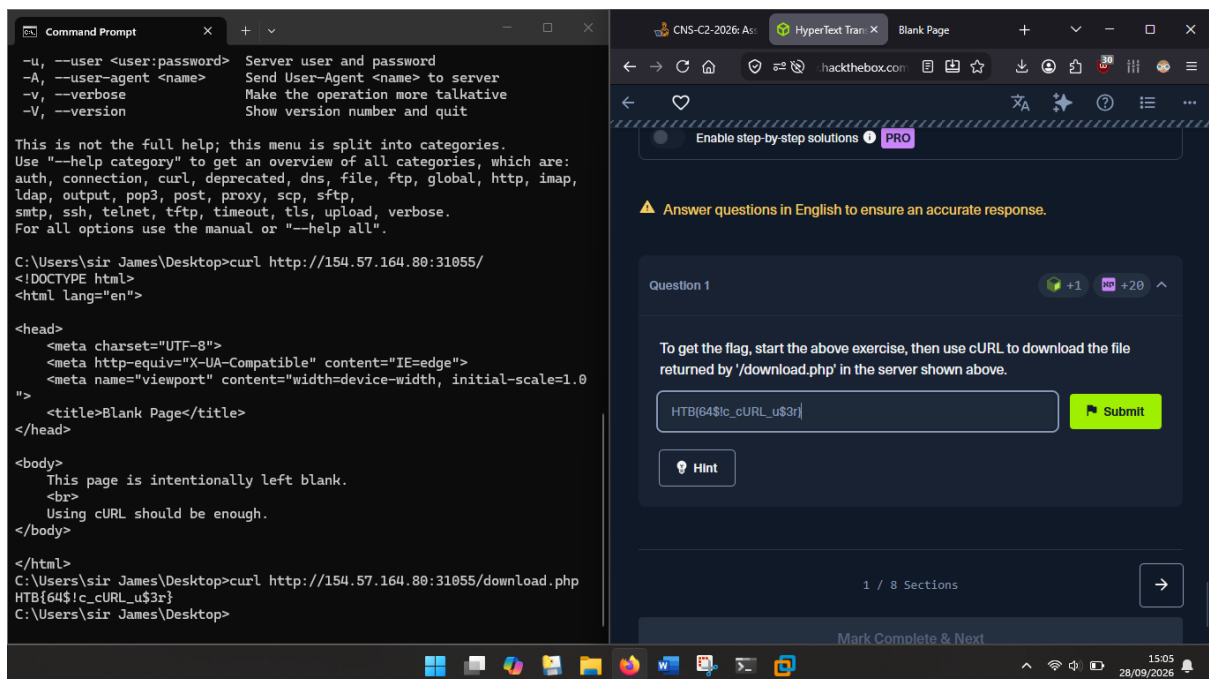
C:\Users\sir James\Desktop>curl -s -O http://info.cern.ch/index.html

C:\Users\sir James\Desktop>curl -h
Usage: curl [options...] <url>
-d, --data <data> HTTP POST data
-f, --fail Fail fast with no output on HTTP errors
-h, --help <category> Get help for commands
-i, --include Include response headers in output
-o, --output <file> Write to file instead of stdout
-O, --remote-name Write output to file named as remote file
-s, --silent Silent mode
-T, --upload-file <file> Transfer local FILE to destination
-u, --user <user:password> Server user and password
-A, --user-agent <name> Send User-Agent <name> to server
-v, --verbose Make the operation more talkative
-V, --version Show version number and quit

This is not the full help; this menu is split into categories.
Use "--help category" to get an overview of all categories, which are:
auth, connection, curl, deprecated, dns, file, ftp, global, http, imap, ldap, output, pop3, post, proxy, scp, sftp,
smtp, ssh, telnet, tftp, timeout, tls, upload, verbose.
For all options use the manual or "--help all".

C:\Users\sir James\Desktop>
```

Quiz: Get the flag



```
C:\Users\sir James\Desktop>curl http://154.57.164.80:31055/
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0
">
  <title>Blank Page</title>
</head>

<body>
  This page is intentionally left blank.
  <br>
  Using cURL should be enough.
</body>

</html>
C:\Users\sir James\Desktop>curl http://154.57.164.80:31055/download.php
HTB{64$c_cURL_u$3r}
```

cURL for HTTPS

cURL checks for an invalid or outdated SSL certificate before proceeding.

if we ever contact a website with an invalid SSL certificate or an outdated one, then cURL by default would not proceed with the communication to protect against the earlier mentioned MITM attacks:

-k flag

To skip the certificate check with cURL, we can use the -k flag: useful when testing local web application or with a web application hosted for practice purposes

```
Command Prompt
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>curl https://inlane freight.com

C:\Users\sir James\Desktop>curl -k https://www.inlane freight.com
<!DOCTYPE html>
<html lang="en-US">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="profile" href="http://gmpg.org/xfn/11">
<title>Inlane freight &#8211; Protected by Wordfence</title>
<link rel="dns-prefetch" href="//fonts.googleapis.com" />
<link rel="dns-prefetch" href="//s.w.org" />
<link rel="alternate" type="application/rss+xml" title="Inlane freight &#8211; Feed" href="https://www.inlane freight.com/index.php/feed/" />
<link rel="alternate" type="application/rss+xml" title="Inlane freight &#8211; Comments Feed" href="https://www.inlane freight.com/index.php/comments/feed/" />
<script type="text/javascript">
window._wpemojiSettings = {"baseUrl":"https://s.w.org/images/core/emoji/13.0.1/72x72/", "ext":".png", "svgUrl":"https://s.w.org/images/core/emoji/13.0.1/svg/", "svgExt":".svg", "source":{"concatemoji":"https://www.inlane freight.com/wp-includes/js/wp-emoji-release.min.js?ver=5.6.21"}};
!function(e,a,t){var n,r,o,i=a.createElement("canvas"),p=i.getContext&&i.getContext("2d");function s(e,t){var a=String.fromCharCode;return p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,a,0),0,0);e=i.toDataURL();return p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,t,0,0),0,0);function c(e){var t=a.createElement("script");t.src=e,t.defer=t.type="text/javascript",a.getElementsByTagName("head")[0].appendChild(t)}for(o=Array("flag","emoji"),t.supports=everything:!0,everythingExceptFlag:!0),r=0;r<=o.length;r++){t.supports[o[r]]=function(e){if(!p||p.fillText)return!1;switch(p.textBaseline="top",p.font="600 32px Arial",e){case"flag":return s([127987,65039,8205,9895,65039],[127987,65039,8203,9895,65039]);!1:s([55356,56826,55356,56819],[55356,56826,8203,55356,56819])&&s([55356,57332,56128,56423,56128,56418,56128,56430,56128,56423,8203,56128,56423,8203,56128,56418,8203,56128,56421,8203,56128,56430,8203,56128,56423,8203,56128,56417]);case"emoji":return s([55357,56424,8205,55356,57212],[55357,56424,8203,55356,57212])}return!1}(o[r]),t.supports.everything=t.supports.everything&&t.supports[o[r]],"Flag"!==o[r]&&(t.supports.everythingExceptFlag=t.supports.everythingExceptFlag&&t.supports[o[r]]);t.supports.everythingExceptFlag=t.supports.everythingExceptFlag&&t.supports.flag,t.DOMReady=!1,t.readyCallback=function(){t.DOMReady=!0,t.supports.everything||(n=function(){t.readyCallback()},a.addEventListener("DOMContentLoaded",n,!1),e.addEventListener("load",n,!1)):e.attachEvent("onreadystatechange",function){n.readyState&&t.readyCallback()}}),n=t.source||{}).concatemoji?c(n.concatemoji):n.wpemoji&&n.twemoji&&c(n.twemoji),c(n.wpemoji)}}(window,document,window._wpemojiSettings);
</script>
```

HTTP Requests and Responses

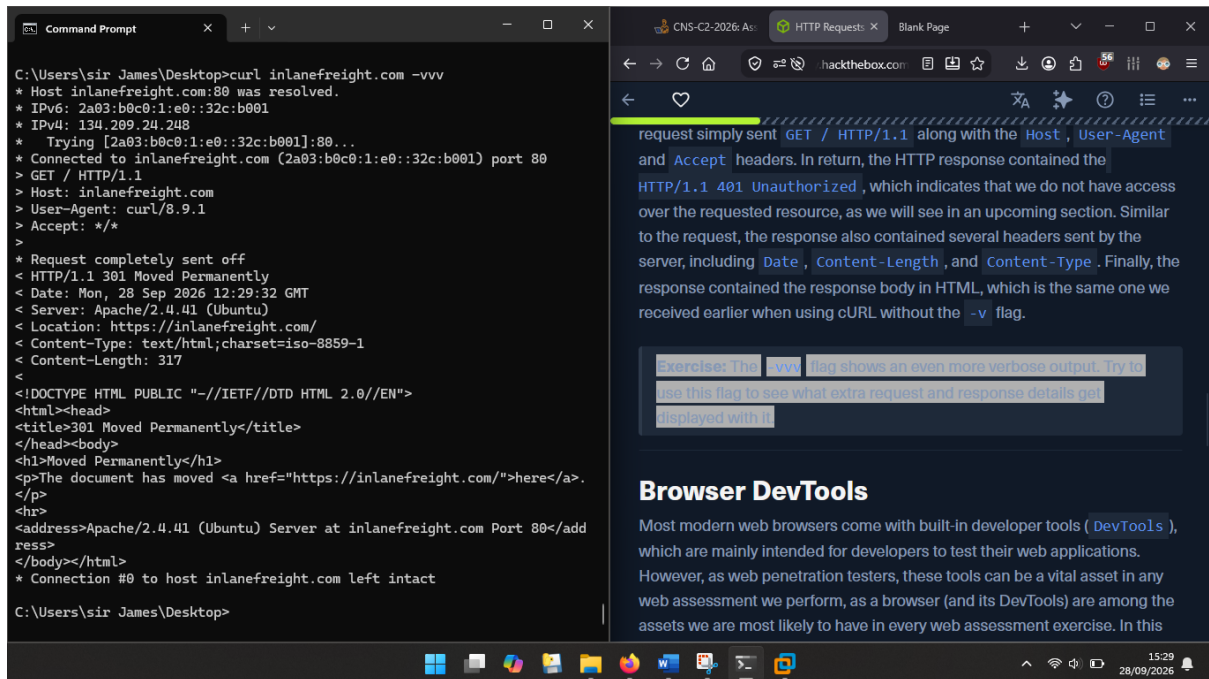
To view the full HTTP request and response, we can simply add the -v verbose flag to our earlier commands, and it should print both the request and response:

```
Command Prompt
<address>Apache/2.4.41 (Ubuntu) Server at inlane freight.com Port 80</address>
</body></html>
* Connection #0 to host inlane freight.com left intact

C:\Users\sir James\Desktop>curl inlane freight.com -v
* Host inlane freight.com:80 was resolved.
* IPv6: 2a03:b0c0:1:e0::32c:b001
* IPv4: 134.209.24.248
* Trying [2a03:b0c0:1:e0::32c:b001]:80...
* Connected to inlane freight.com (2a03:b0c0:1:e0::32c:b001) port 80
> GET / HTTP/1.1
> Host: inlane freight.com
> User-Agent: curl/8.9.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Date: Mon, 28 Sep 2026 12:26:40 GMT
< Server: Apache/2.4.41 (Ubuntu)
< Location: https://inlane freight.com/
< Content-Type: text/html; charset=iso-8859-1
< Content-Length: 317
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="https://inlane freight.com/">here</a>.</p>
</body>
</html>
<address>Apache/2.4.41 (Ubuntu) Server at inlane freight.com Port 80</address>
</body></html>
* Connection #0 to host inlane freight.com left intact

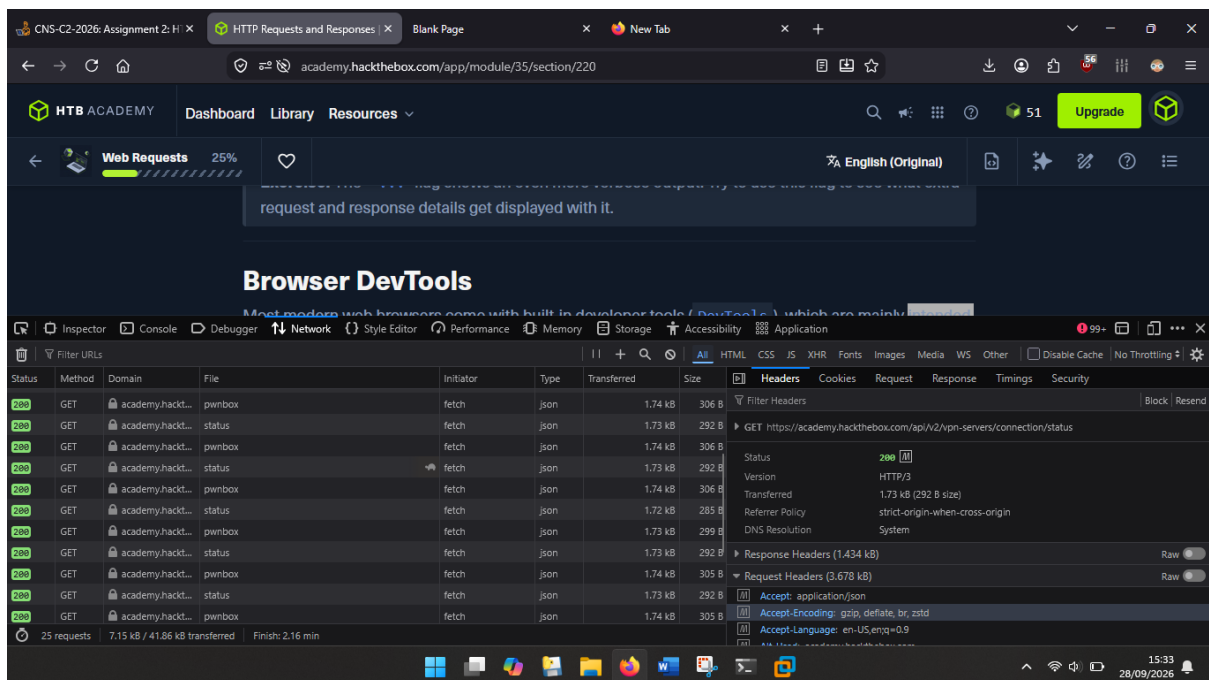
C:\Users\sir James\Desktop>
```

Exercise: The -vvv flag shows an even more verbose output. Try to use this flag to see what extra request and response details get displayed with it.



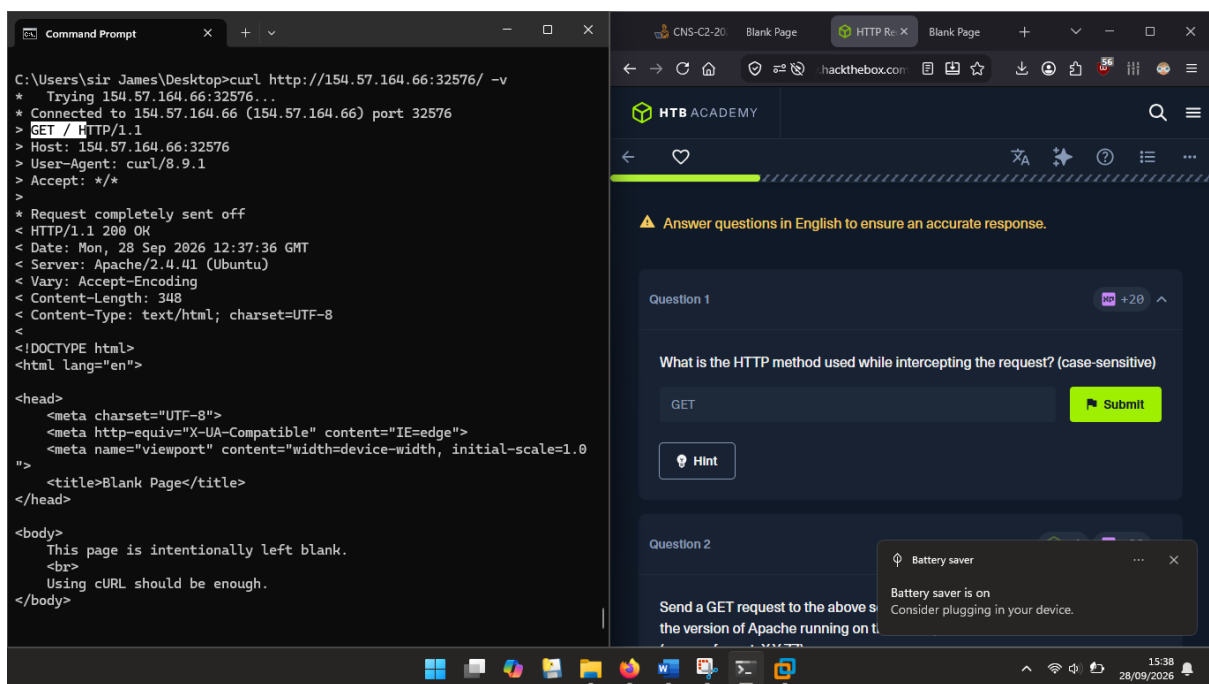
Browser DevTools

Exercise: Try clicking on any of the requests to view their details. You can then click on the Response tab to view the response body, and then click on the Raw button to view the raw (unrendered) source code of the response body.

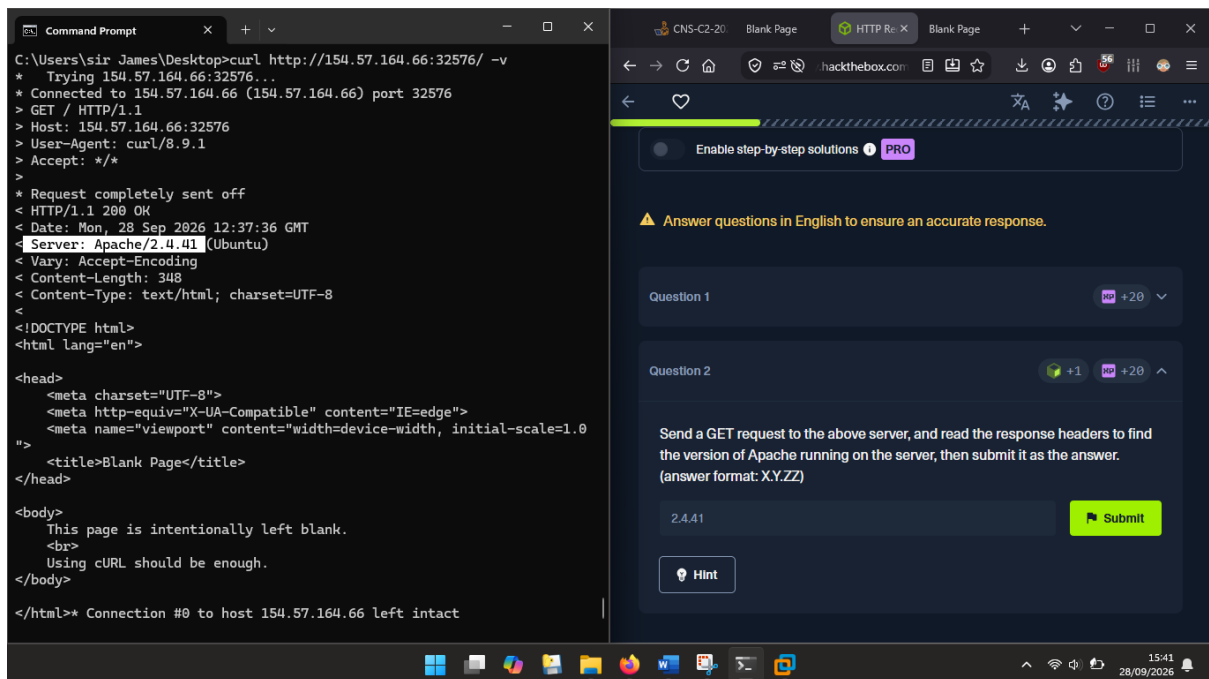


Quiz

What is the HTTP method used while intercepting the request? GET



Send a GET request to the above server, and read the response headers to find the version of Apache running on the server, then submit it as the answer: 2.4.41



HTTP Headers

General Headers

General headers are used in both HTTP requests and responses. They are contextual and are used to describe the message rather than its contents. Eg Date: Wed, 16 Feb 2022 10:38:44 GMT, Connection: close

Entity Headers

These headers are used to describe the content (entity) transferred by a message. They are usually found in responses and POST or PUT requests. Eg Content-Type: text/html, Media-Type: application/pdf, boundary="b4e4fbd93540", Content-Length: 385, Content-Encoding: gzip

Request Headers

The client sends Request Headers in an HTTP transaction. These headers are used in an HTTP request and do not relate to the content of the message. The following headers are commonly seen in HTTP requests. Host: www.inlanefreight.com, User-Agent: curl/7.77.0, Cookie: PHPSESSID=b4e4fbd93540

Response Headers

Response Headers can be used in an HTTP response and do not relate to the content. Certain response headers such as Age, Location, and Server are used to provide more context about the response. The following headers are commonly seen in HTTP responses. Eg Server: Apache/2.2.14 (Win32), Set-Cookie: PHPSESSID=b4e4fbd93540, WWW-Authenticate: BASIC realm="localhost"

Security Headers

Finally, we have Security Headers. With the increase in the variety of browsers and web-based attacks, defining certain headers that enhanced security was necessary. HTTP Security headers are a class of response headers used to specify certain rules and policies to be followed by the browser while accessing the website. Eg Content-Security-Policy: script-src 'self', Strict-Transport-Security: max-age=31536000, Referrer-Policy: origin

cURL response headers

If we were only interested in seeing the response headers, then we can use the -I flag to send a HEAD request and only display the response headers. Furthermore, we can use the -i flag to display both the headers and the response body (e.g. HTML code). The difference between the two is that -I sends a HEAD request (as will see in the next section), while -i sends any request we specify and prints the headers as well.

```
Command Prompt
Using curl should be enough.
</body>

</html>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>
C:\Users\sir James\Desktop>curl -I https://www.inlanefreight.com
HTTP/1.1 200 OK
Date: Mon, 28 Sep 2026 12:59:20 GMT
Server: Apache/2.4.41 (Ubuntu)
Link: <https://www.inlanefreight.com/index.php/wp-json/>; rel="https://api.w.org/"
Link: <https://www.inlanefreight.com/index.php/wp-json/wp/v2/pages/7>; rel="alternate"; type="application/json"
Link: <https://www.inlanefreight.com/>; rel=shortlink
Content-Type: text/html; charset=UTF-8
```

```
Command Prompt

C:\Users\sir James\Desktop>curl -i https://www.inlanefreight.com
HTTP/1.1 200 OK
Date: Mon, 28 Sep 2026 15:31:12 GMT
Server: Apache/2.4.41 (Ubuntu)
Link: <https://www.inlanefreight.com/index.php/wp-json/>; rel="https://api.w.org/"
Link: <https://www.inlanefreight.com/index.php/wp-json/wp/v2/pages/7>; rel="alternate"; type="application/json"
Link: <https://www.inlanefreight.com/>; rel=shortlink
Accept-Encoding:
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="en-US">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="profile" href="http://gmpg.org/xfn/11">
<title>Inlanefreight &#8211; Protected by Wordfence</title>
<link rel='dns-prefetch' href='//fonts.googleapis.com' />
<link rel='dns-prefetch' href='//s.w.org' />
<link rel="alternate" type="application/rss+xml" title="Inlanefreight &#8211; Feed" href="https://www.inlanefreight.com/index.php/feed/" />
<link rel="alternate" type="application/rss+xml" title="Inlanefreight &#8211; Comments Feed" href="https://www.inlanefreight.com/index.php/comments/feed/" />

<script type="text/javascript">
window.wpemojiSettings = {"baseUrl": "https://s.w.org/images/core/emoji/13.0.1/72x72/", "ext": ".png", "svgUrl": "https://s.w.org/images/core/emoji/13.0.1/svg/", "svgExt": ".svg", "source": {"concatemoji": "https://www.inlanefreight.com/wp-includes/js/wp-emoji-release.min.js?ver=5.6.21"}};
!function(e,a,t){var n,r,o,i=a.createElement("canvas"),p=i.getContext&&i.getContext("2d");function s(e,t){var a=String.fromCharCode; p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,e),0,0);e=i.toDataURL();return p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,t),0,0),e===i.toDataURL()}function c(e){var t=a.createElement("script");t.src=e,t.defer=t.type="text/javascript",a.getElementsByTagName("head")[0].appendChild(t)}for(o=Array("flag","emoji"),t.supports={everything:!0,everythingExceptFlag:!0},r=0;r<o.length;r++){t.supports[o[r]]=function(e){c(p)!p.fillText)return!1;switch(p.textBaseline="top",p.font="600 32px Arial",e){case"flag":return s([127987,65039,9895,65039],!1)!s([55356,56826,55356,56819],[55356,56826,8203,55356,56819])&&!s([55356,57332,56128,56423,56128,56418,56128,56421,56128,56430,5
```

On above images we have headers

Date: Mon, 28 Sep 2026 15:31:12 GMT; general header

Content-Type: text/html; charset=UTF-8 entity header

Server: Apache/2.4.41 (Ubuntu); response header

Link: <https://www.inlanefreight.com/>; rel=shortlink; response header

Transfer-Encoding: chunked; general header

use the -A to set our User-Agent, as follows

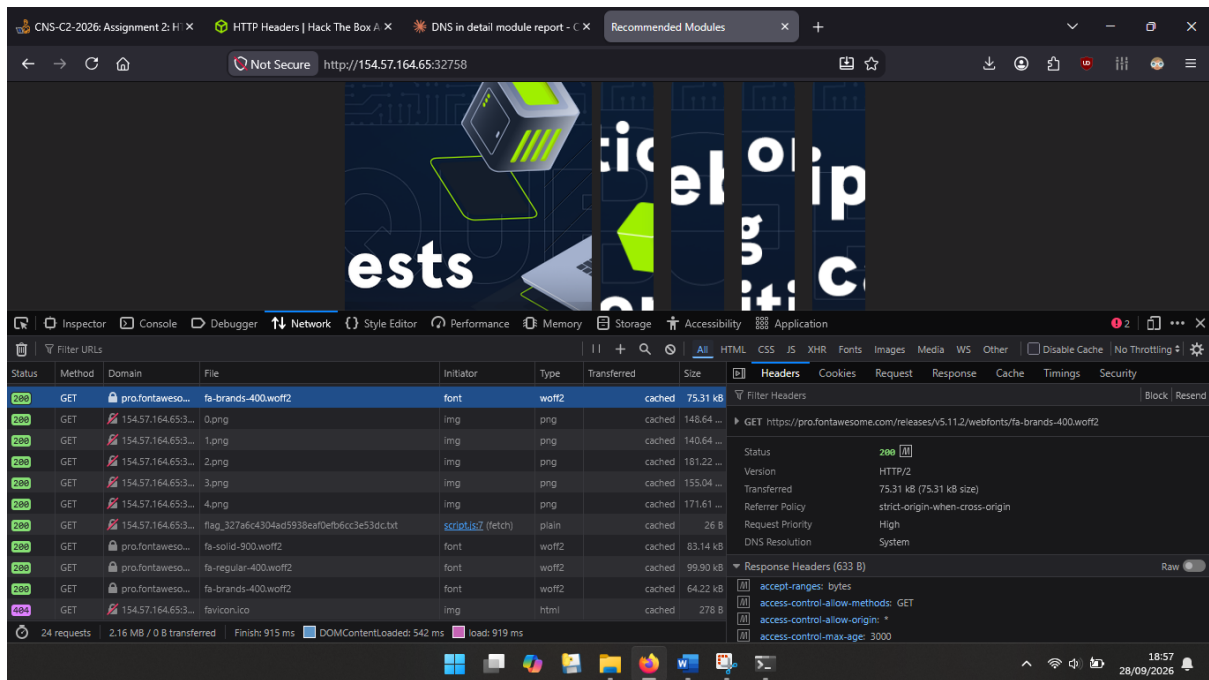
Exercise: Try to use the `-I` or the `-v` flags with the above example, to ensure that we did change our User-Agent with the `-A` flag.

```
Command Prompt
C:\Users\sir James\Desktop>curl https://www.inlanefreight.com -A 'Mozilla/5.0' -v
* Host www.inlanefreight.com:443 was resolved.
* IPv6: (none)
* IPv4: 134.209.24.248
* Trying 134.209.24.248:443...
* Connected to www.inlanefreight.com (134.209.24.248) port 443
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* using HTTP/1.x
> GET / HTTP/1.1
> Host: www.inlanefreight.com
> User-Agent: 'Mozilla/5.0'
> Accept: */*
>
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: failed to decrypt data, need more data
< HTTP/1.1 200 OK
< Date: Mon, 28 Sep 2026 15:50:30 GMT
< Server: Apache/2.4.41 (Ubuntu)
< Link: <https://www.inlanefreight.com/index.php/wp-json/>; rel="https://api.w.org/"
< Link: <https://www.inlanefreight.com/index.php/wp-json/wp/v2/pages/7>; rel="alternate"; type="application/json"
< Link: <https://www.inlanefreight.com/>; rel=shortlink
< Vary: Accept-Encoding
< Transfer-Encoding: chunked
< Content-Type: text/html; charset=UTF-8
<
<!DOCTYPE html>
<html lang="en-US">
```

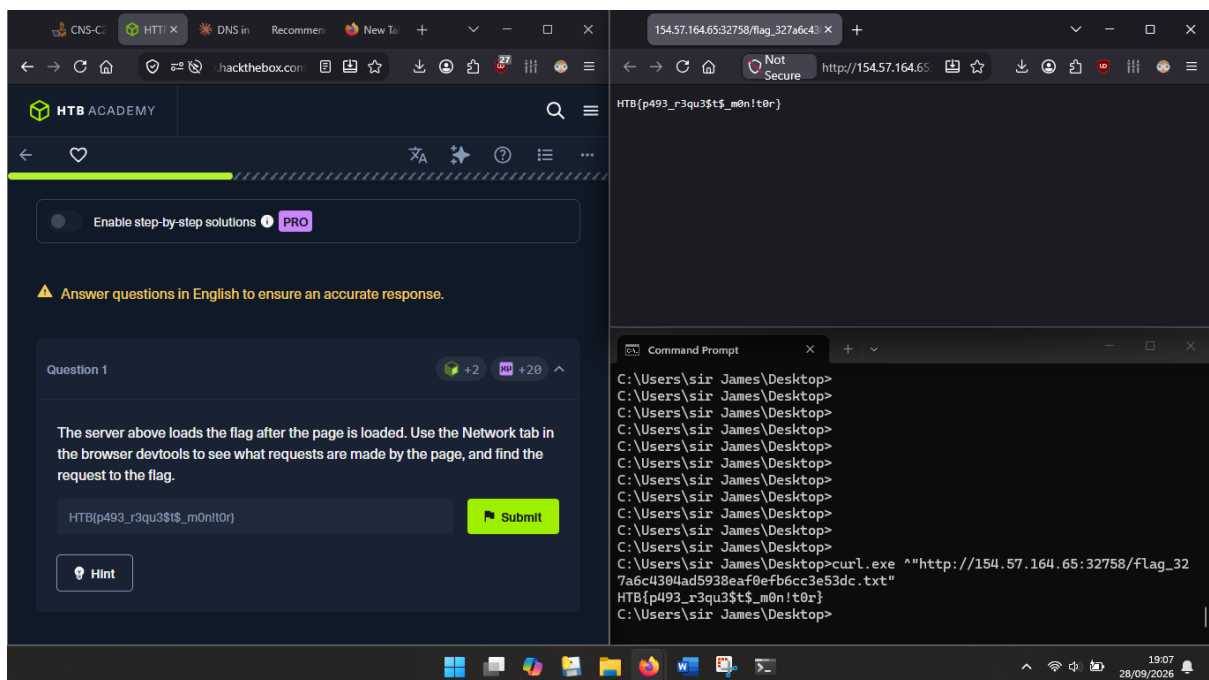
Using the `-A` command we have observed that we can redefine user agent

Browser DevTools

In the first Headers tab, we see both the HTTP request and HTTP response headers. The devtools automatically arrange the headers into sections, but we can click on the Raw button to view their details in their raw format. Furthermore, we can check the Cookies tab to see any cookies used by the request, as discussed in an upcoming section.



Using DevTools, we located the flag in the response:



Part 2: HTTP Methods

HTTP Methods and Codes

- GET: retrieve a resource. Parameters go in the URL.
- POST: send data to the server (forms, logins, uploads). Data goes in the body.

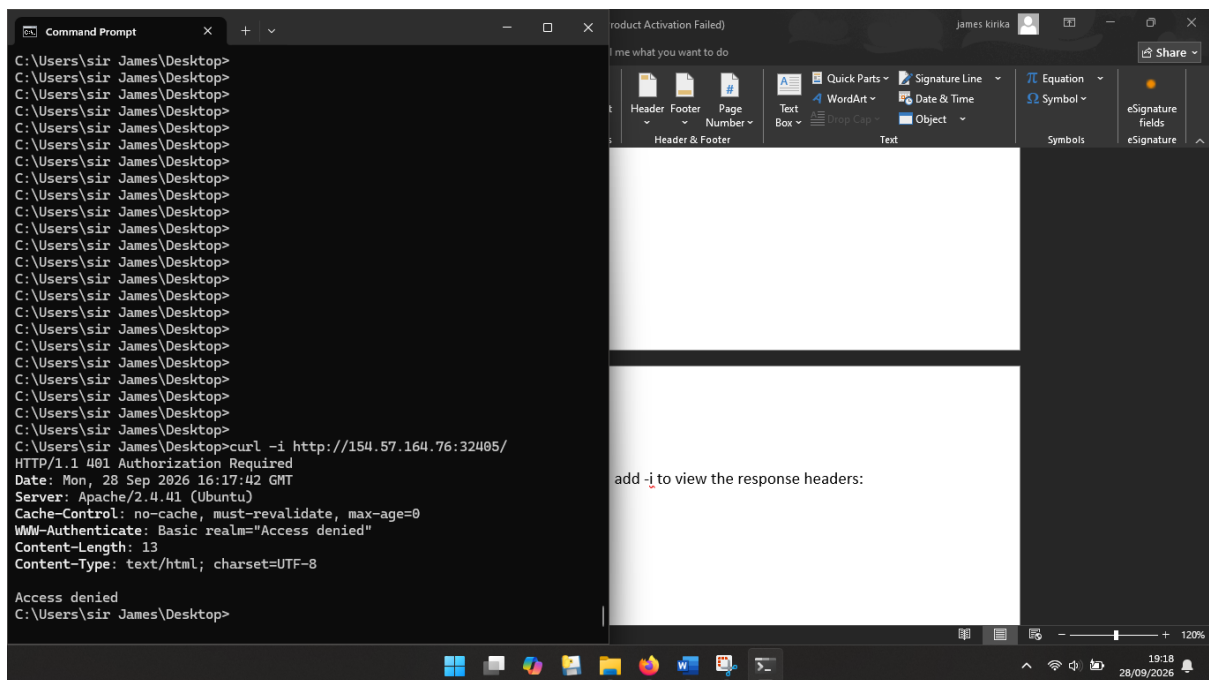
- HEAD: same as GET, but returns headers only, no body.
- PUT: create or replace a resource at the given location.
- DELETE: remove the specified resource.
- OPTIONS: ask the server which methods it supports for a resource.
- PATCH: apply a partial modification to a resource.

Status Codes

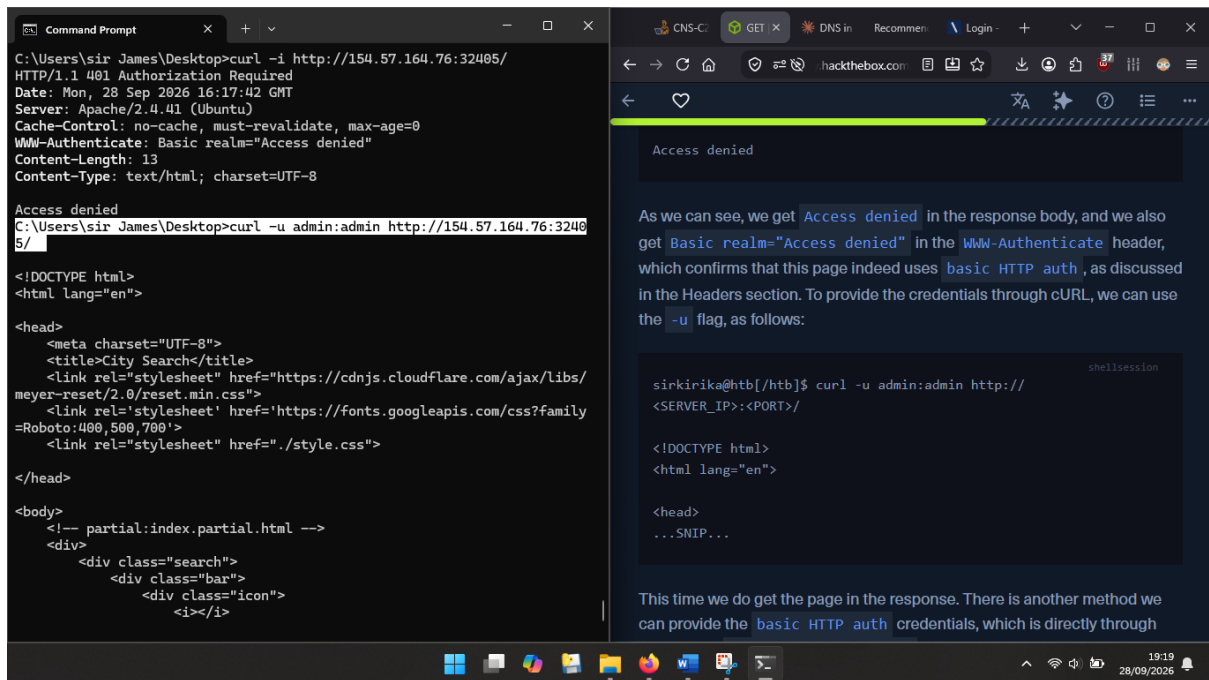
- 1xx (Informational): request received, processing continues. Rarely seen in practice.
- 2xx (Success): request succeeded.
- 3xx (Redirection): client must take another step.
- 4xx (Client error): the request is at fault.
- 5xx (Server error): the server failed.

HTTP Basic Auth

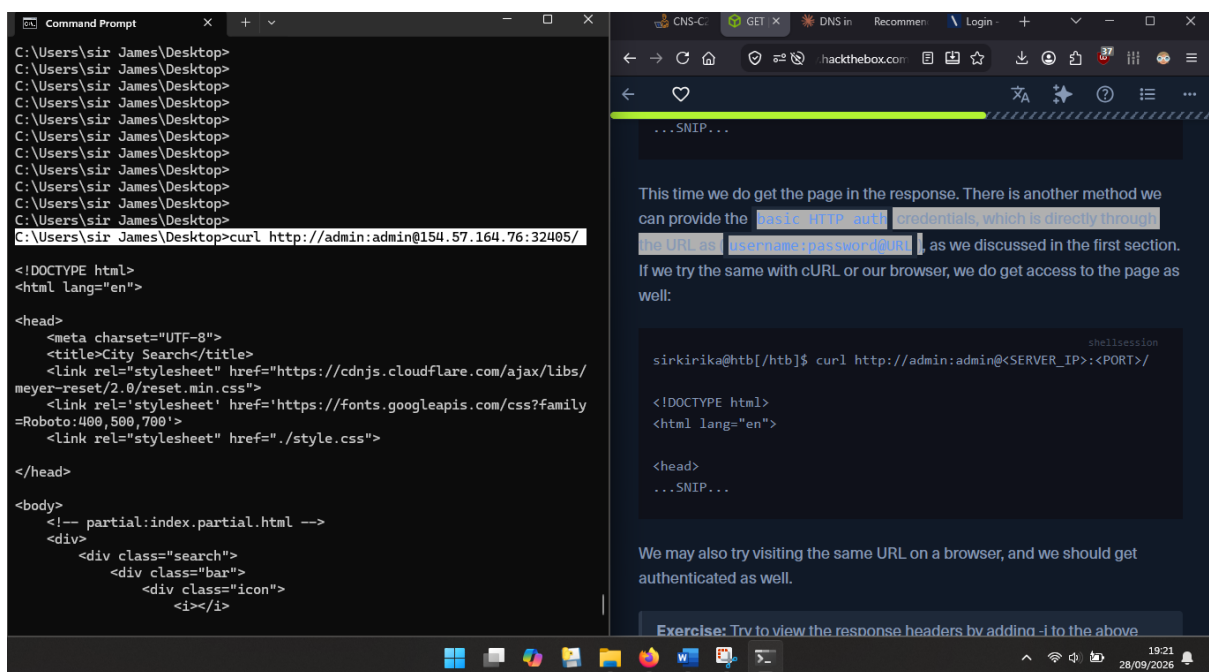
Let's try to access the page with cURL, and we'll add -i to view the response headers:



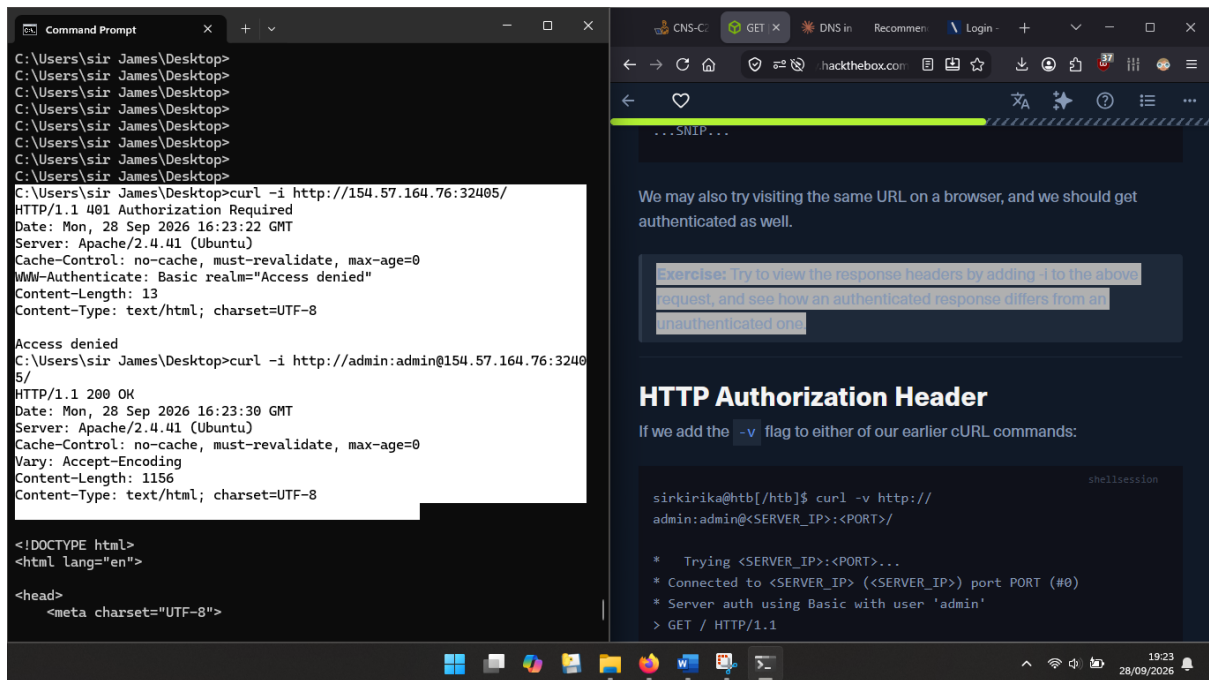
To provide the credentials through cURL, we can use the `-u` flag, as follows:



We can also supply Basic Auth credentials directly through the URL as `username:password@URL`.



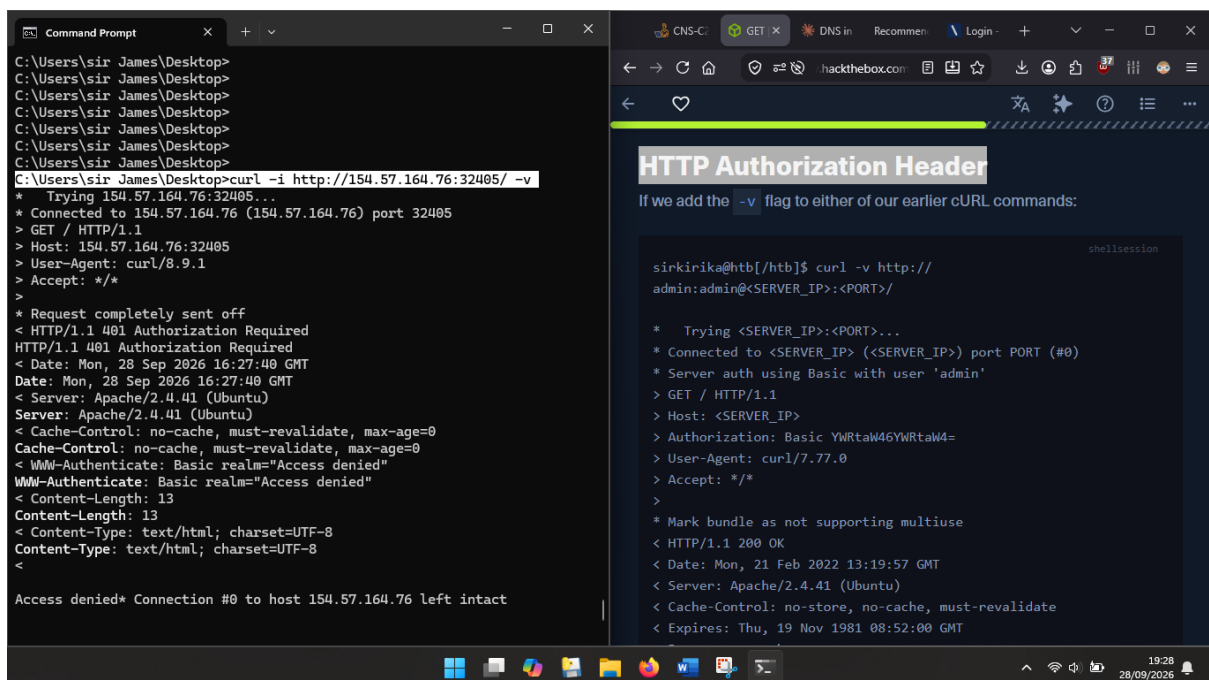
Exercise: Try to view the response headers by adding `-i` to the above request, and see how an authenticated response differs from an unauthenticated one.

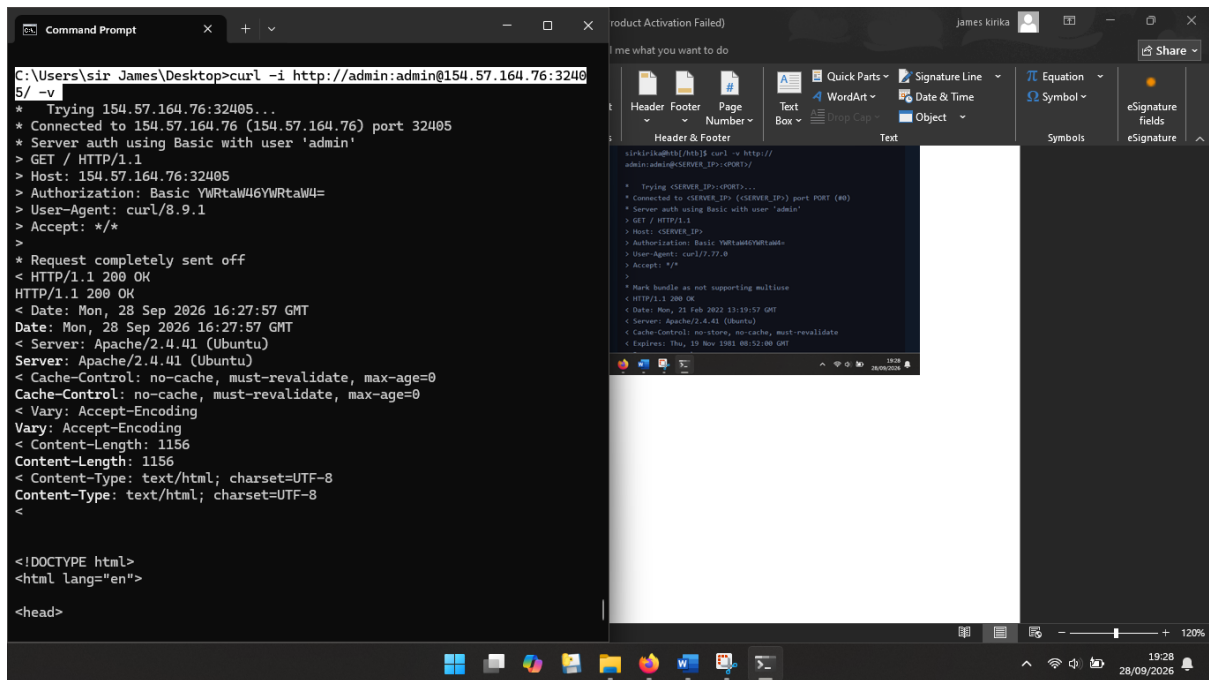


The headers in both cases are almost similar apart from Content-Length and www-Authenticate header is missing in the authenticated one.

HTTP Authorization Header

Adding -v command

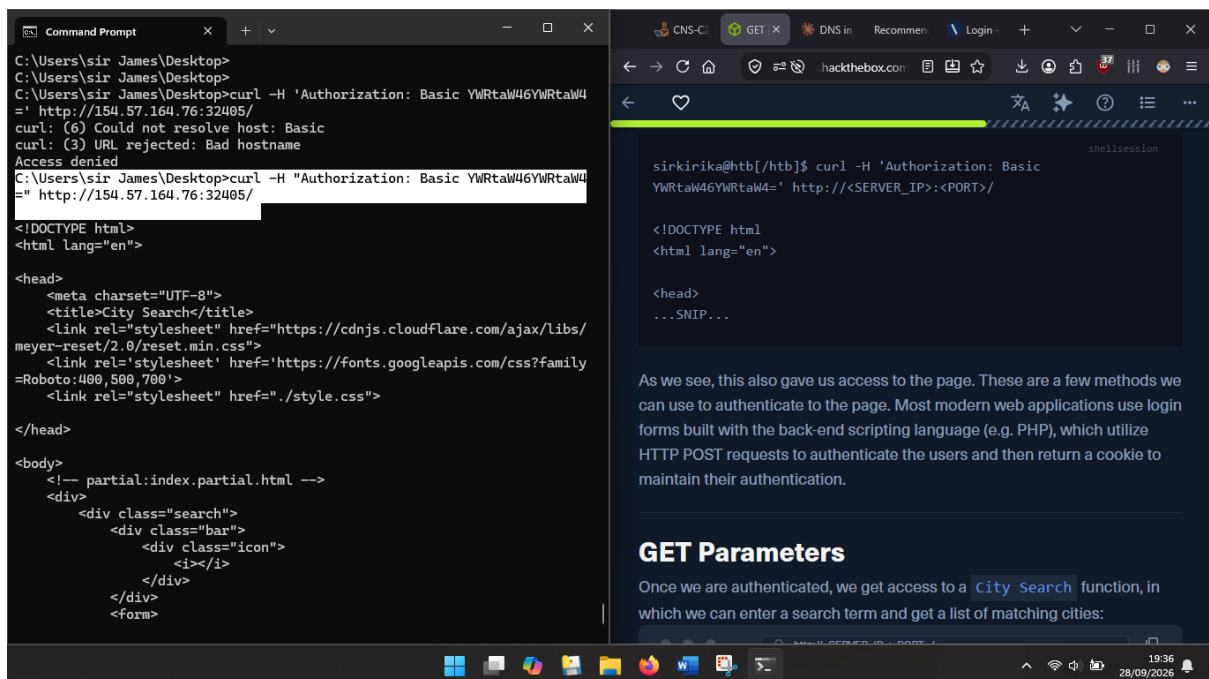




As we are using basic HTTP auth, we see that our HTTP request sets the Authorization header to Basic YWRtaW46YWRtaW4=, which is the base64 encoded value of admin:admin. If we were using a modern method of authentication (e.g. JWT), the Authorization would be of type Bearer and would contain a longer encrypted token.

-H flag

Let's try to manually set the Authorization, without supplying the credentials, to see if it does allow us access to the page. We can set the header with the -H flag, and will use the same value from the above HTTP request. We can add the -H flag multiple times to specify multiple headers:



GET Parameters

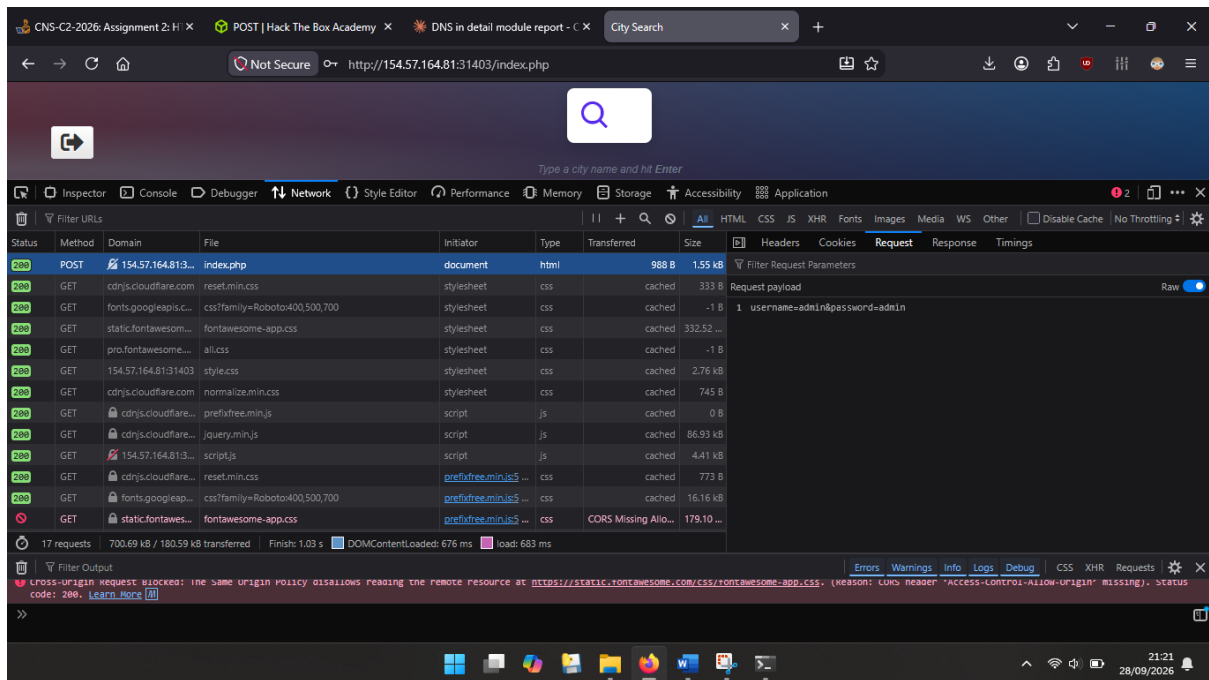
The screenshot displays a Windows desktop environment. The left window is a Command Prompt titled "Command Prompt". It shows the following commands and their outputs:

```
C:\Users\sir James\Desktop> curl http://154.57.164.76:32405/search.php?search=le -H "Authorization: Basic YWRtaWw4GjYWRtaWw4="
curl: (6) Could not resolve host: search
curl: (3) URL rejected: Bad hostname
curl: (6) Could not resolve host: le

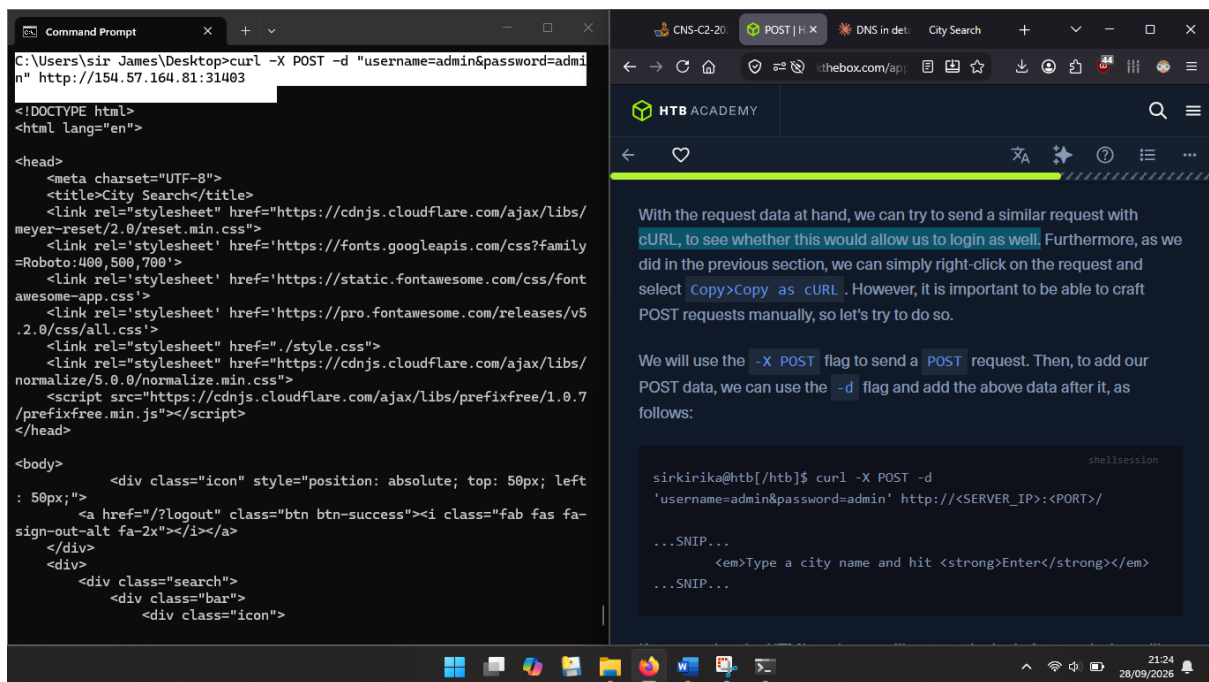
C:\Users\sir James\Desktop> curl "http://154.57.164.76:32405/search.php?search=le" -H "Authorization: Basic YWRtaWw4GjYWRtaWw4="
Leeds (UK)
Dudley (UK)
Leicester (UK)
Newcastle (UK)
Los Angeles (US)
Jacksonville (US)
Seattle (US)
Nashville-Davidson (US)
```

The right window is a web browser titled "GET x" displaying the results of the search query. The page has a dark theme and shows a list of cities: Leeds (UK), Dudley (UK), Leicester (UK), Newcastle (UK), Los Angeles (US), Jacksonville (US), Seattle (US), and Nashville-Davidson (US). A red box highlights the first three items: Leeds (UK), Dudley (UK), and Leicester (UK).

In DevTools, we can view the POST request's raw data:

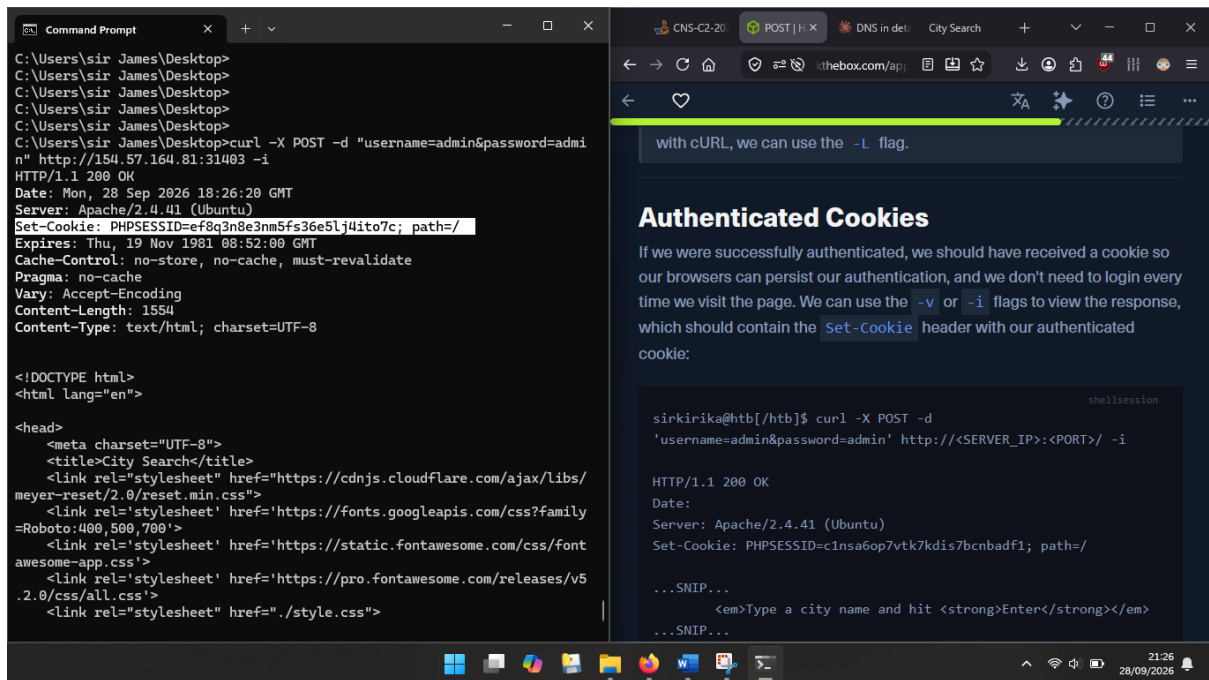


We repeated this with cURL to see whether it would also allow us to log in.

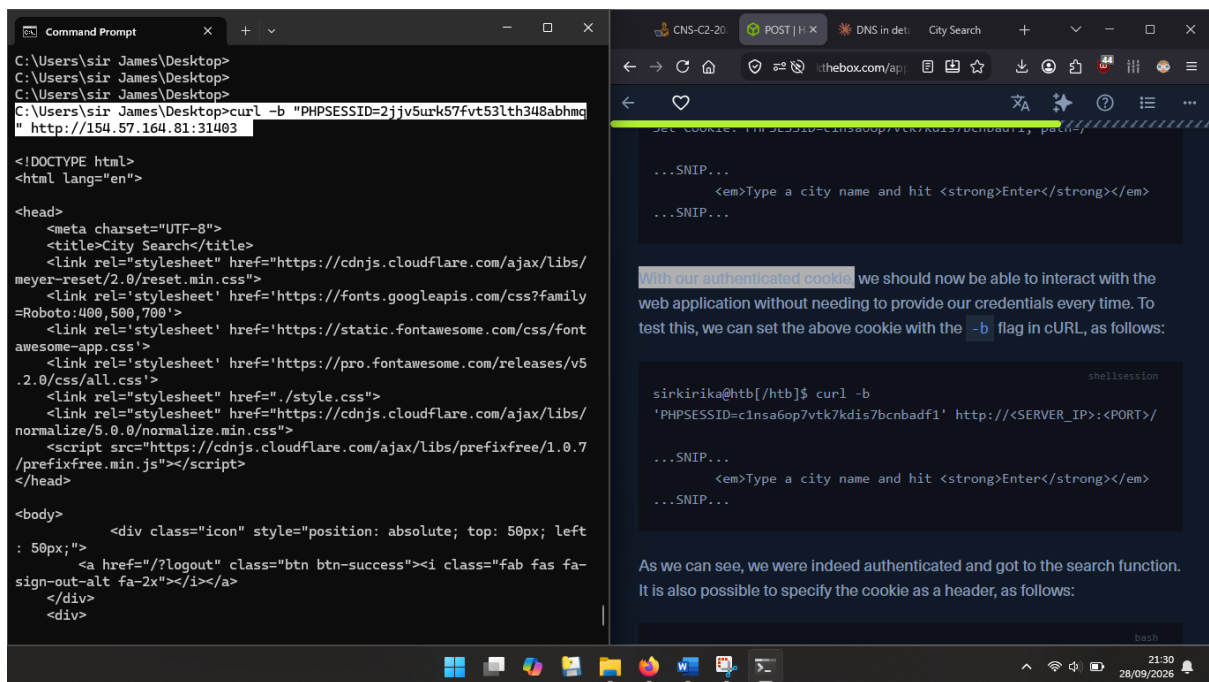


Authenticated Cookies

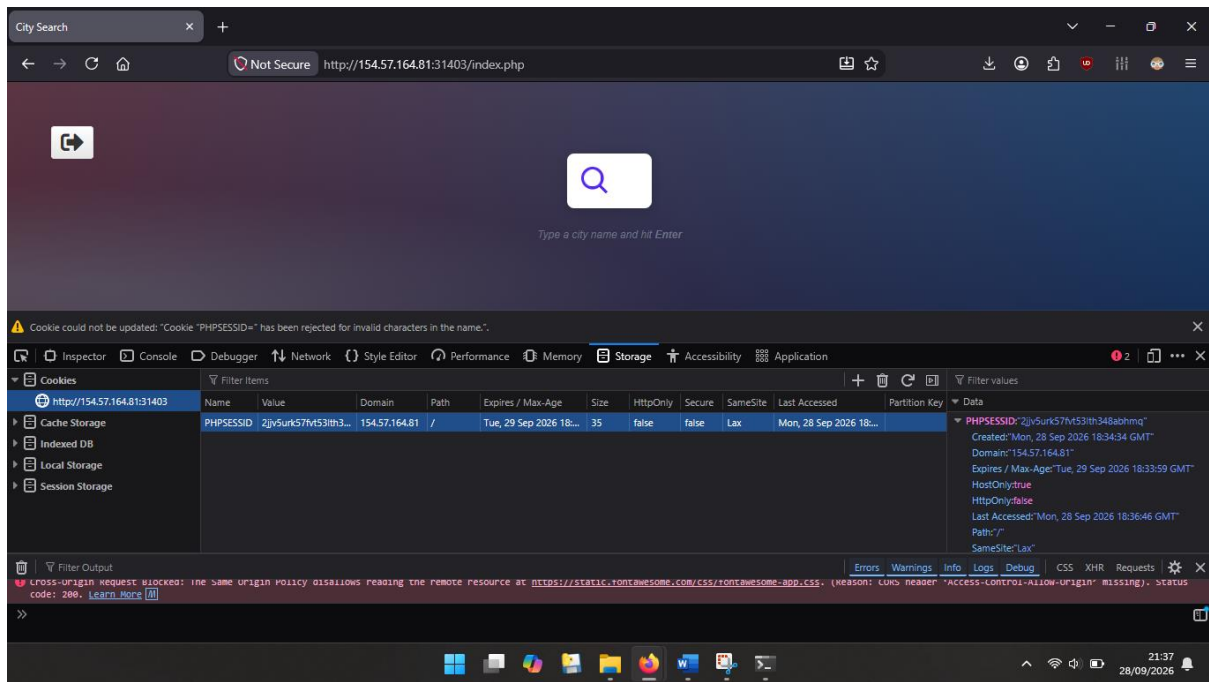
If we were successfully authenticated, we should have received a cookie so our browsers can persist our authentication, and we don't need to login every time we visit the page. We can use the -v or -i flags to view the response, which should contain the Set-Cookie header with our authenticated cookie:



We then used the authenticated cookie to log in via cURL:

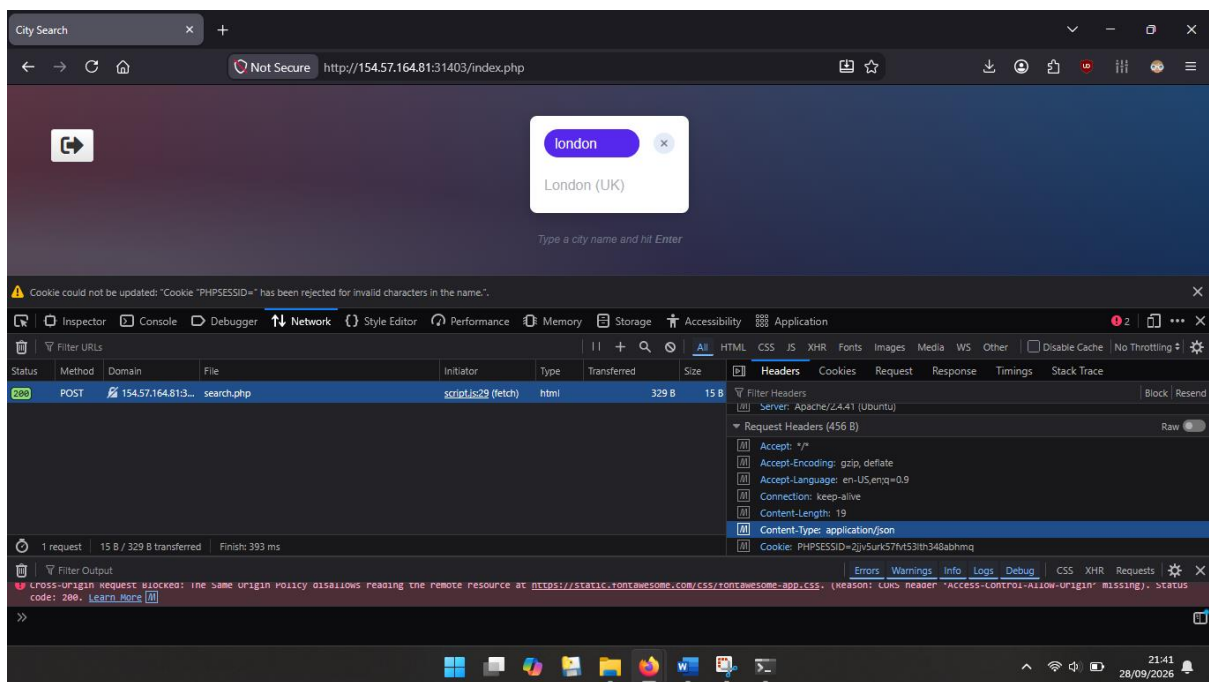


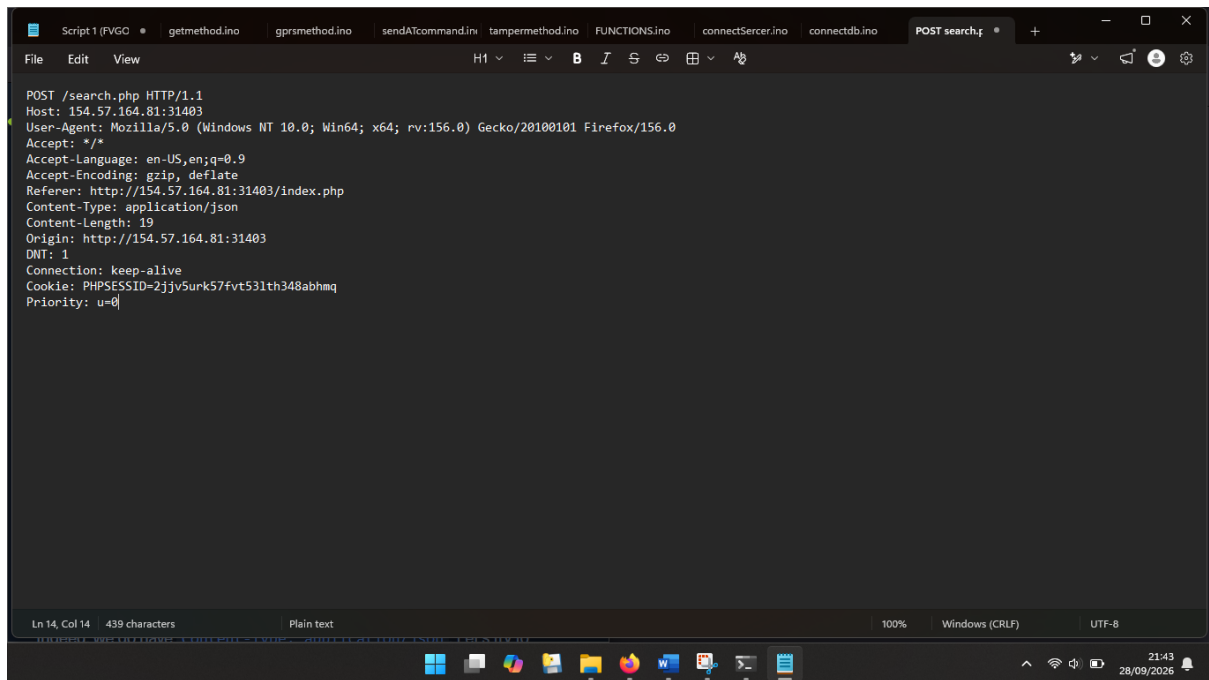
We repeated this in the browser using the same cookie header:



JSON Data

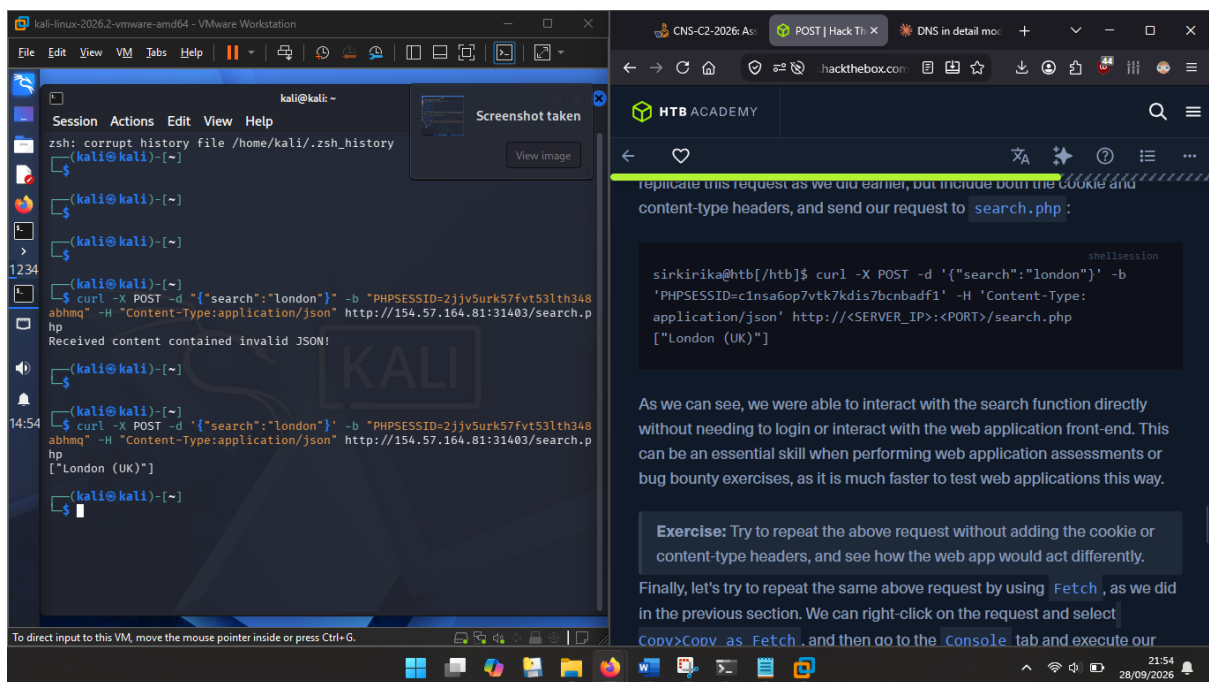
The POST data appear to be in JSON format, so our request must have specified the Content-Type header to be application/json.





```
Script 1 (FVGC) • getmethod.ino gprsmethod.ino sendATcommand.ino tampermethod.ino FUNCTIONS.ino connectSercer.ino connectdb.ino POST search.f
File Edit View H1 B I S E T A
POST /search.php HTTP/1.1
Host: 154.57.164.81:31403
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101 Firefox/156.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Referer: http://154.57.164.81:31403/index.php
Content-Type: application/json
Content-Length: 19
Origin: http://154.57.164.81:31403
DNT: 1
Connection: keep-alive
Cookie: PHPSESSID=2jjv5urk57fvt53lth348abhmq
Priority: u=0
Ln 14, Col 14 439 characters Plain text 100% Windows (CRLF) UTF-8
```

We replicated this request as we did earlier, but included both the cookie and Content-Type headers, sending our request to search.php.



The image shows a Kali Linux terminal window on the left and a web browser window on the right. The terminal displays a series of commands and their outputs. The first command is a curl POST request to search.php with a JSON body and a cookie. The output is "Received content contained invalid JSON!". The second command is a curl POST request to search.php with a JSON body and a cookie, and the output is ["London (UK)"]. The web browser window on the right shows a page from HTB Academy with a search bar and a search button. The search bar contains the text "replicate this request as we did earlier, but include both the cookie and content-type headers, and send our request to search.php:". The search button is labeled "Search".

```
kali@kali: ~  
zsh: corrupt history file /home/kali/.zsh_history  
$ curl -X POST -d '{"search":"london"}' -b 'PHPSESSID=2jjv5urk57fvt53lth348abhmq' -H 'Content-Type:application/json' http://154.57.164.81:31403/search.php  
Received content contained invalid JSON!  
$ curl -X POST -d '{"search":"london"}' -b 'PHPSESSID=2jjv5urk57fvt53lth348abhmq' -H 'Content-Type:application/json' http://154.57.164.81:31403/search.php  
["London (UK)"]
```

HTB ACADEMY

replicate this request as we did earlier, but include both the cookie and content-type headers, and send our request to search.php :

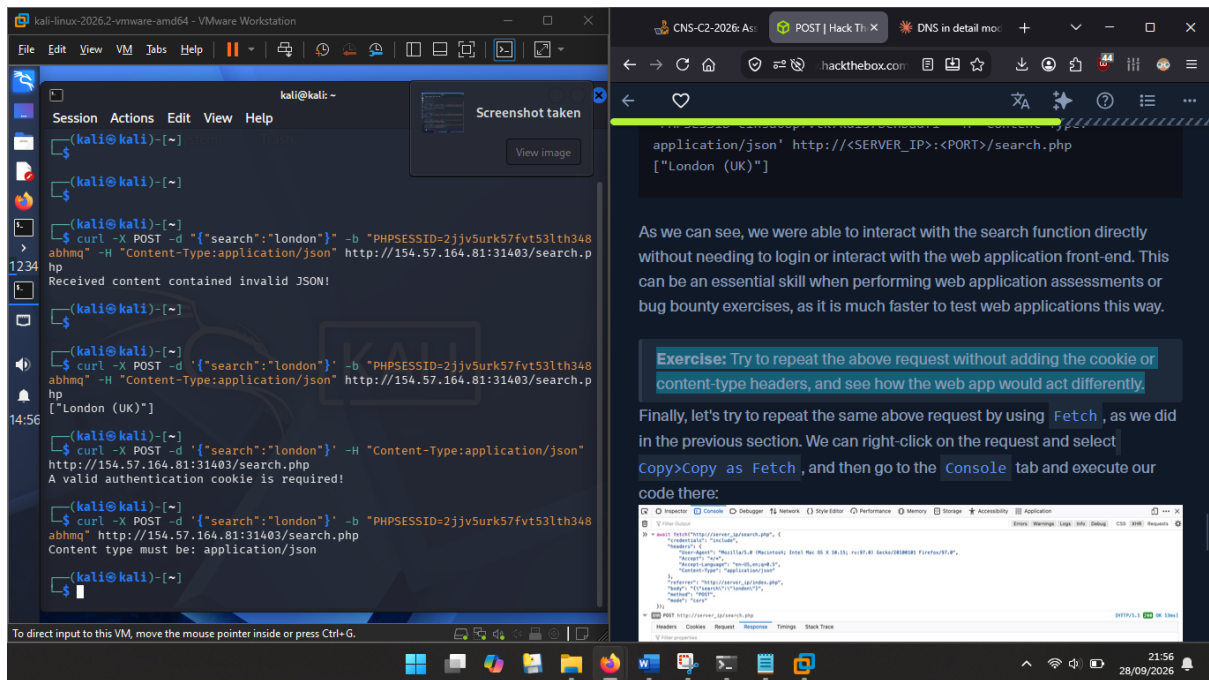
```
sirkirika@htb[/htb]$ curl -X POST -d '{"search":"london"}' -b 'PHPSESSID=c1nsa6op7vtk7kdis7bcnbadf1' -H 'Content-Type: application/json' http://<SERVER_IP>:<PORT>/search.php  
["London (UK)"]
```

As we can see, we were able to interact with the search function directly without needing to login or interact with the web application front-end. This can be an essential skill when performing web application assessments or bug bounty exercises, as it is much faster to test web applications this way.

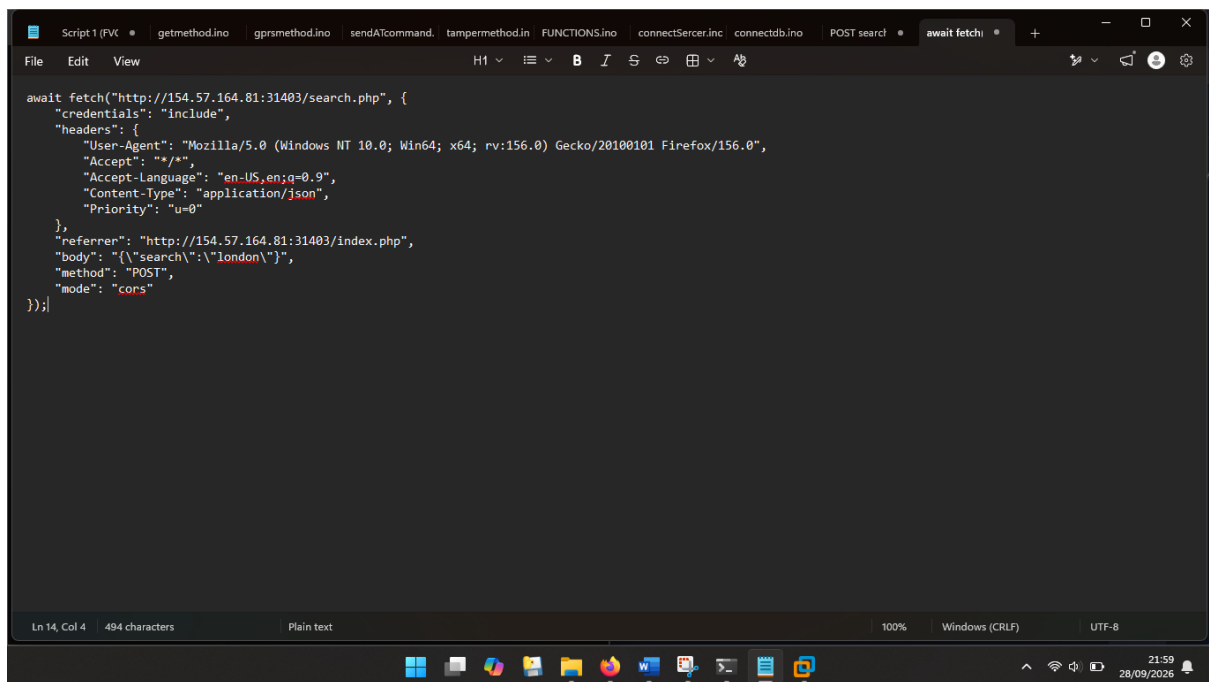
Exercise: Try to repeat the above request without adding the cookie or content-type headers, and see how the web app would act differently.

Finally, let's try to repeat the same above request by using `Fetch`, as we did in the previous section. We can right-click on the request and select `Copy>Copy as Fetch`, and then go to the `Console` tab and execute our

Exercise: Try to repeat the above request without adding the cookie or content-type headers, and see how the web app would act differently.



We repeated the same request using Fetch in the browser Console:



Executing fetch on the console

The screenshot shows a Kali Linux VM with a browser window open to a HackTheBox page. The browser console displays an error: "Content-Security-Policy: The page's settings blocked the loading of a resource (connect-src) directive: 'connect-src https:'". The error message indicates that the browser blocked the loading of a resource because the page's settings only allow outbound requests over https, but the target is plain http://. The web page shows an exercise instruction: "Exercise: Try to repeat the above request without adding the cookie or content-type headers, and see how the web app would act differently. Finally, let's try to repeat the same above request by using Fetch, as we did in the previous section. We can right-click on the request and select Copy>Copy as Fetch, and then go to the Console tab and execute our code there:". The console shows a successful fetch request to http://154.57.164.81:31403/search.php, returning a JSON response with city information.

The Console of a page has a CSP allowing outbound requests (connect-src) only over https:.. My target is plain http://, so the browser blocked it before it left.

Quiz

The screenshot shows a Kali Linux VM with a terminal window open. The terminal displays a series of curl commands and their outputs. The first command is a POST request to http://154.57.164.81:31403/search.php with a JSON body {"search": "london"} and a PHPSESSID cookie. The output is a JSON response with city information. The second command is a POST request to the same URL with a JSON body {"search": "flag"} and the same PHPSESSID cookie. The output is a JSON response with a flag. The third command is a POST request to the same URL with a JSON body {"search": "flag"} and the same PHPSESSID cookie. The output is a JSON response with a flag. The web page shows a quiz question: "Obtain a session cookie through a valid login, and then use the cookie with cURL to search for the flag through a JSON POST request to /search.php". The question is marked as "Question 1" and has a "Submit" button. The answer field contains the text "HTB{p0\$t_r3p34t3r}" and the "Submit" button is highlighted in green.

CRUD API

The main API operations on a requested database entity include:

Create → POST: adds the specified data to the database table.

Read → GET: reads the specified entity from the database table.

Update → PUT: updates the data of the specified database table.

Delete → DELETE: removes the specified row from the database table.

Read

We started by reading city entries from the API with a GET request:

The screenshot shows a Kali Linux terminal window on the left and a web browser window on the right. The terminal displays a series of curl commands and their outputs. The first command is a POST request to /search.php with a search term of 'london'. The second command is a POST request to /search.php with a search term of 'flag'. The third command is a GET request to /api.php/city/london. The output of the third command is a 404 Not Found error. The fourth command is a GET request to /api.php/city/london with a Content-Type header of application/json. The output of the fourth command is a JSON string: [{"city_name": "London", "country_name": "(UK)"}]. The web browser window on the right shows the HTB Academy website with a 'Read' section. The text in the 'Read' section explains that the first thing we will do when interacting with an API is reading data. It mentions that we can specify the table name after the API (e.g. /city) and then specify our search term (e.g. /london). It also shows a curl command and its output, which is a JSON string. The text explains that the result is sent as a JSON string and that we can pipe the output to the jq utility to format it properly. It also mentions that we will silence any unneeded curl output with -s.

```
kali@kali:~$ curl -X POST -d '{"search": "london"}' -b "PHPSESSID=2jjv5urk57fvt53lth348abhmq" http://154.57.164.81:31403/search.php
Content type must be: application/json

(kali@kali)-[~]
$ curl -X POST -d '{"search": "flag"}' -b "PHPSESSID=2jjv5urk57fvt53lth348abhmq" http://154.57.164.81:31403/search.php
Content type must be: application/json

(kali@kali)-[~]
$ curl -X POST -d '{"search": "flag"}' -b "PHPSESSID=2jjv5urk57fvt53lth348abhmq" -H "Content-Type:application/json" http://154.57.164.81:31403/search.php
["flag: HTB{p0$t_r3p34t3r}"]

(kali@kali)-[~]
$ curl http://154.57.164.81:31403/api.php/city/london
<DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN>
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at 154.57.164.81 Port 31403</address>
</body></html>

(kali@kali)-[~]
$ curl http://154.57.164.69:31284/api.php/city/london
[{"city_name": "London", "country_name": "(UK)"}]

(kali@kali)-[~]
$
```

Read

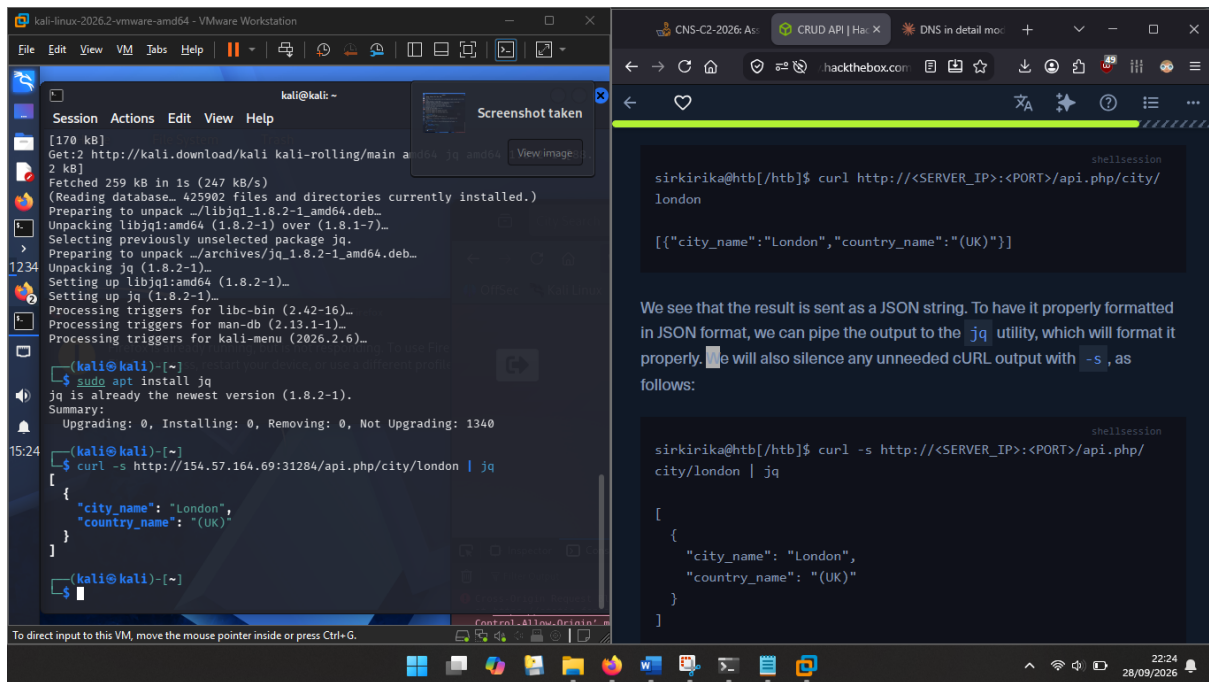
The first thing we will do when interacting with an API is reading data. As mentioned earlier, we can simply specify the table name after the API (e.g. /city) and then specify our search term (e.g. /london), as follows:

```
sirkirika@htb[htb]$ curl http://<SERVER_IP>:<PORT>/api.php/city/london

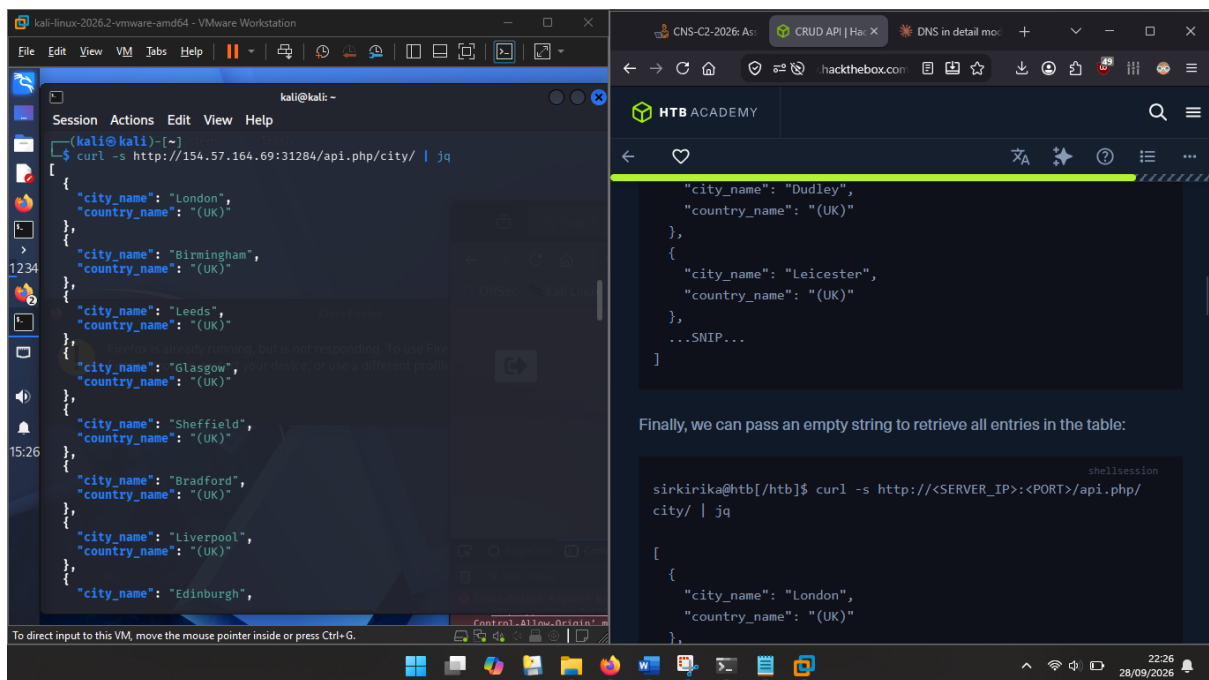
[{"city_name": "London", "country_name": "(UK)"}]
```

We see that the result is sent as a JSON string. To have it properly formatted in JSON format, we can pipe the output to the `jq` utility, which will format it properly. We will also silence any unneeded curl output with `-s`, as follows:

We formatted the output with `jq` and silenced the status output with `-s`:

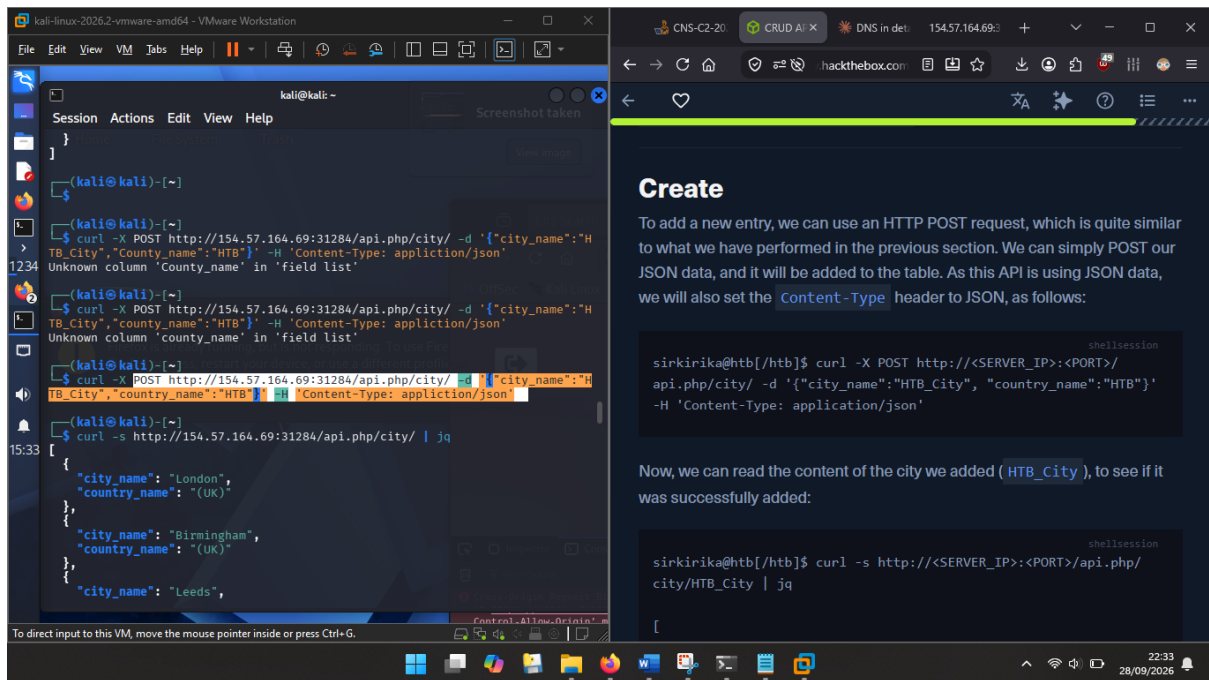


We retrieved all cities in the database by passing an empty string as the city name:

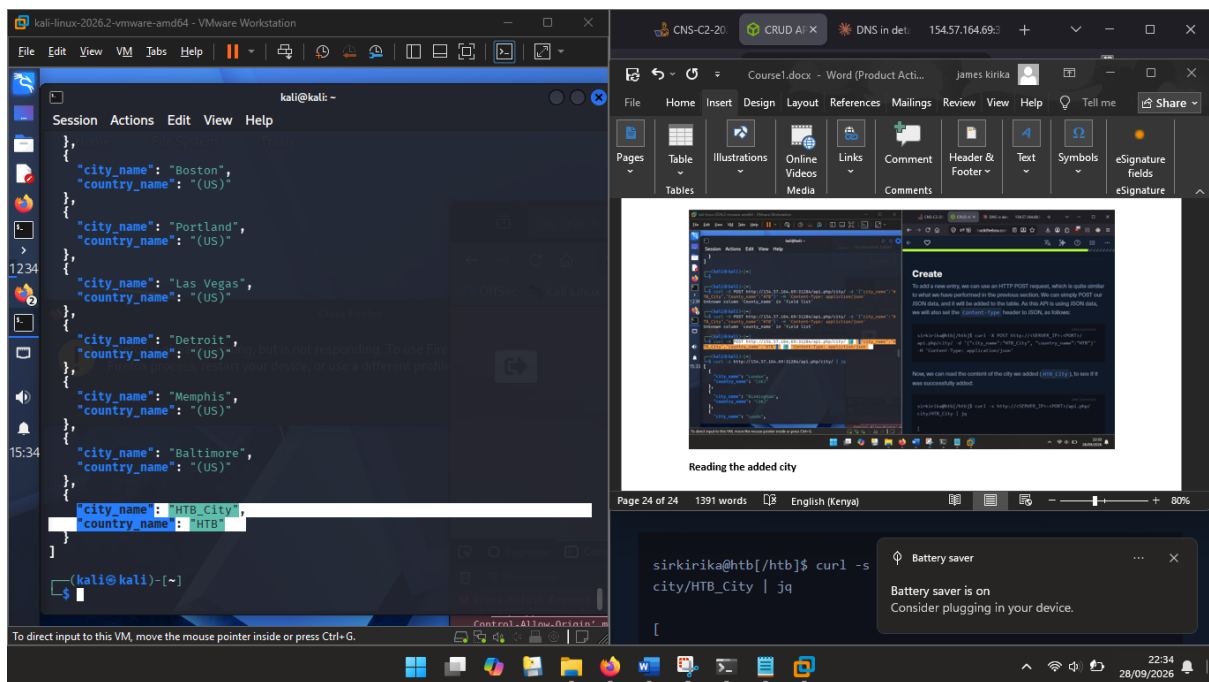


Create

Next, we added a new city with a POST request:

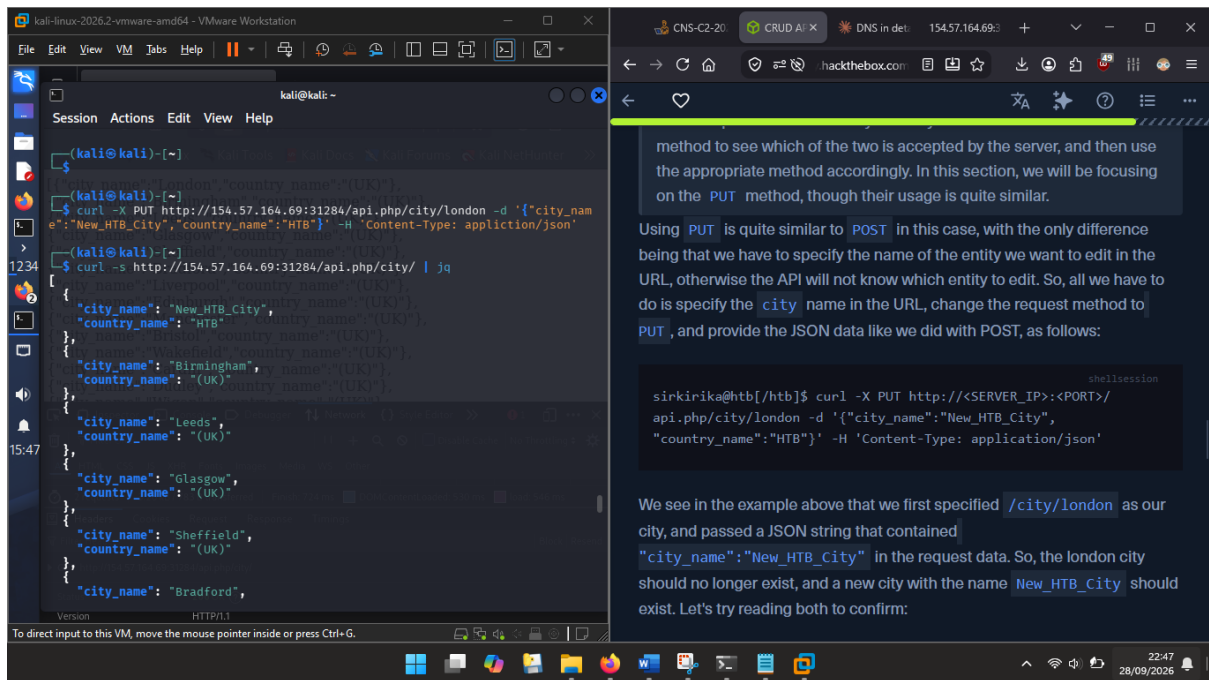


We then confirmed it was added by reading the new city back:

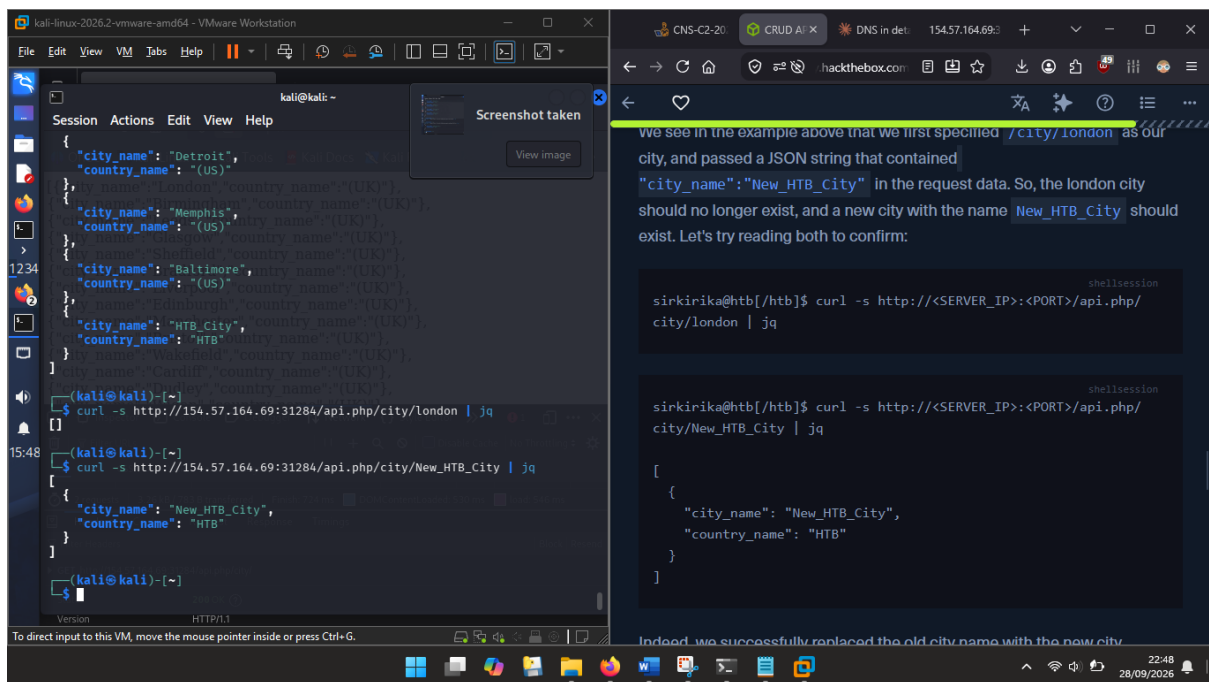


Update

We updated an existing city's name with a PUT request:



The city name London has been replaced with New_HTB_City:



Delete

Finally, we removed a city with a DELETE request:

The screenshot shows a Kali Linux terminal window on the left and a web browser window on the right. The terminal displays a list of cities and the execution of a DELETE request to remove a city named 'New_HTB_City'. The browser window shows a REST client interface with a 'DELETE' method selected, and a text box containing the URL `http://154.57.164.69:31284/api.php/city/New_HTB_City`. Below the text box, there is a section titled 'Exercise: Try to delete any of the cities you added earlier through POST requests, and then read all entries to confirm that they were successfully deleted.'

```
kali@kali:~$ curl -s http://154.57.164.69:31284/api.php/city/London | jq
{
  "city_name": "London", "country_name": "(UK)",
  "city_name": "Birmingham", "country_name": "(UK)",
  "city_name": "HTB_City", "country_name": "(UK)",
  "city_name": "Sheffield", "country_name": "(UK)",
  "city_name": "Bradford", "country_name": "(UK)",
  "city_name": "Dudley", "country_name": "(UK)",
  "city_name": "Bristol", "country_name": "(UK)",
  "city_name": "Leeds", "country_name": "(UK)"
}

kali@kali:~$ curl -s http://154.57.164.69:31284/api.php/city/New_HTB_City | jq
{
  "city_name": "New_HTB_City", "country_name": "HTB"
}

kali@kali:~$ curl -X DELETE http://154.57.164.69:31284/api.php/city/New_HTB_City
[]

kali@kali:~$ curl -s http://154.57.164.69:31284/api.php/city/New_HTB_City | jq
[]
```

Finally, let's try to delete a city, which is as easy as reading a city. We simply specify the city name for the API and use the HTTP **DELETE** method, and it would delete the entry, as follows:

```
sirkirika@htb[/htb]$ curl -X DELETE http://<SERVER_IP>:<PORT>/api.php/city/New_HTB_City
```

As we can see, after we deleted **New_HTB_City**, we get an empty array when we try reading it, meaning it no longer exists.

Exercise: Try to delete any of the cities you added earlier through POST requests, and then read all entries to confirm that they were successfully deleted.

Quiz

First, try to update any city's name to be 'flag'. Then, delete any city. Once done, search for a city named 'flag' to get the flag.

The screenshot shows a Kali Linux terminal window on the left and a web browser window on the right. The terminal displays a list of cities and the execution of a PUT request to update a city named 'Leeds' to 'flag'. The browser window shows a REST client interface with a 'PUT' method selected, and a text box containing the URL `http://154.57.164.69:31284/api.php/city/Leeds`. Below the text box, there is a section titled 'Question 1' with a text input field containing the URL `HTB{crud_4pl_m4n!pu!4t0r}` and a 'Submit' button. Below the input field, there is a 'Hint' button. Below the 'Hint' button, there is a section titled '8 / 8 Sections' with a 'Mark Complete' button.

```
kali@kali:~$ curl -s http://154.57.164.69:31284/api.php/city/Leeds | jq
{
  "city_name": "Leeds", "country_name": "(UK)",
  "city_name": "Birmingham", "country_name": "(UK)",
  "city_name": "HTB_City", "country_name": "(UK)",
  "city_name": "Sheffield", "country_name": "(UK)",
  "city_name": "Bradford", "country_name": "(UK)",
  "city_name": "Dudley", "country_name": "(UK)",
  "city_name": "Bristol", "country_name": "(UK)",
  "city_name": "Leeds", "country_name": "(UK)"
}

kali@kali:~$ curl -X PUT http://154.57.164.69:31284/api.php/city/Leeds
{"city_name": "Leeds", "country_name": "(UK)"}

kali@kali:~$ curl -s http://154.57.164.69:31284/api.php/city/flag | jq
{
  "city_name": "flag", "country_name": "HTB{crud_4pl_m4n!pu!4t0r}"
}
```

First, try to update any city's name to be 'flag'. Then, delete any city. Once done, search for a city named 'flag' to get the flag.

HTB{crud_4pl_m4n!pu!4t0r}

Submit

Hint

8 / 8 Sections

Mark Complete

Conclusion

This module took me from basic cURL requests to authenticating against an API and performing full CRUD operations. The most useful part was seeing the same request three ways: cURL, DevTools Network tab, and a Console fetch, and hitting different failures each time. Shell quoting broke my JSON in cURL, and CSP blocked my fetch over plain HTTP when the page required HTTPS. Working through Basic Auth also made it clear how weak it is on its own, since the Authorization header is just base64, not encryption, and can be replayed by anyone who intercepts it. Overall, the module reinforced that HTTP is inherently plaintext and stateless, and every protection built on top of it, such as HTTPS, secure cookies, CSP, and proper authentication, is something a server or browser has to add deliberately.

Links

[HTB Webrequest](#)

[Linkedin HTB WebRequest](#)